



SoC Architecture and Interfaces

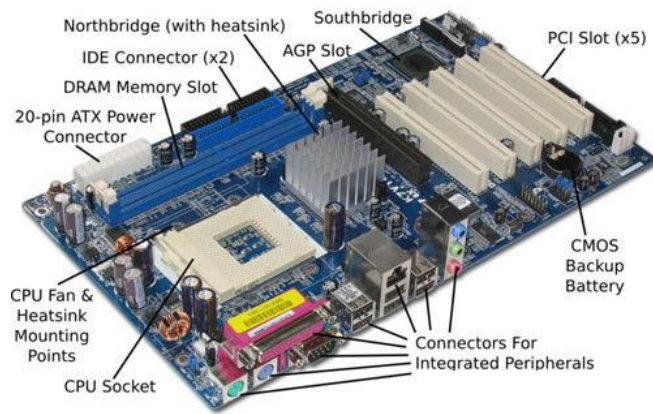
Attribution: Several slides are taken from ARM's open-source course on SoC Architecture and Design

Aman Arora

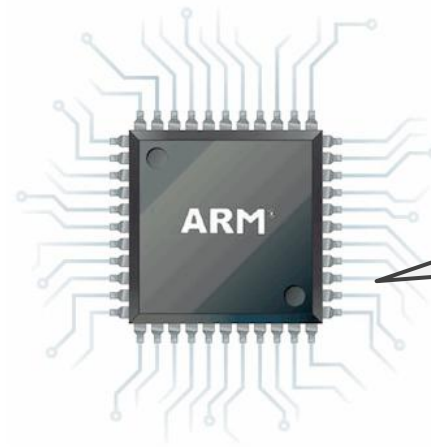
Introduction

What Is an SoC?

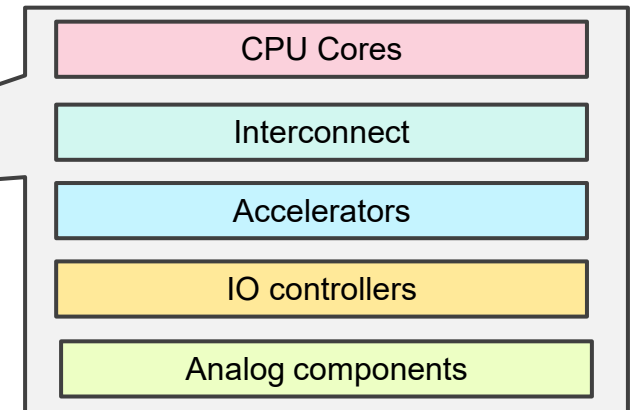
- An SoC is an integrated circuit that packages multiple computing components into a single chip.
- An SoC **may have most or even all** of the components to power a computer.



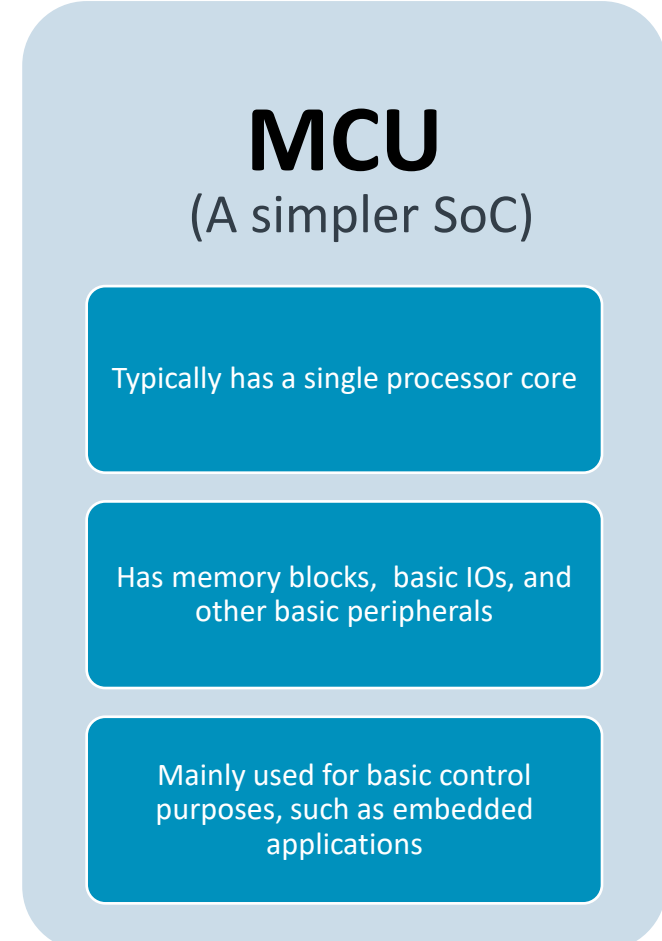
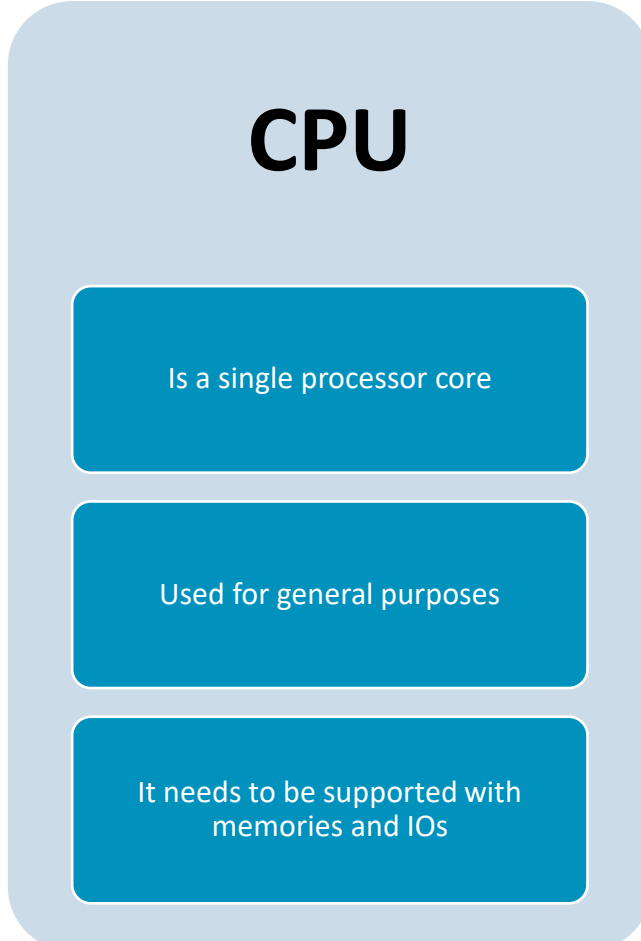
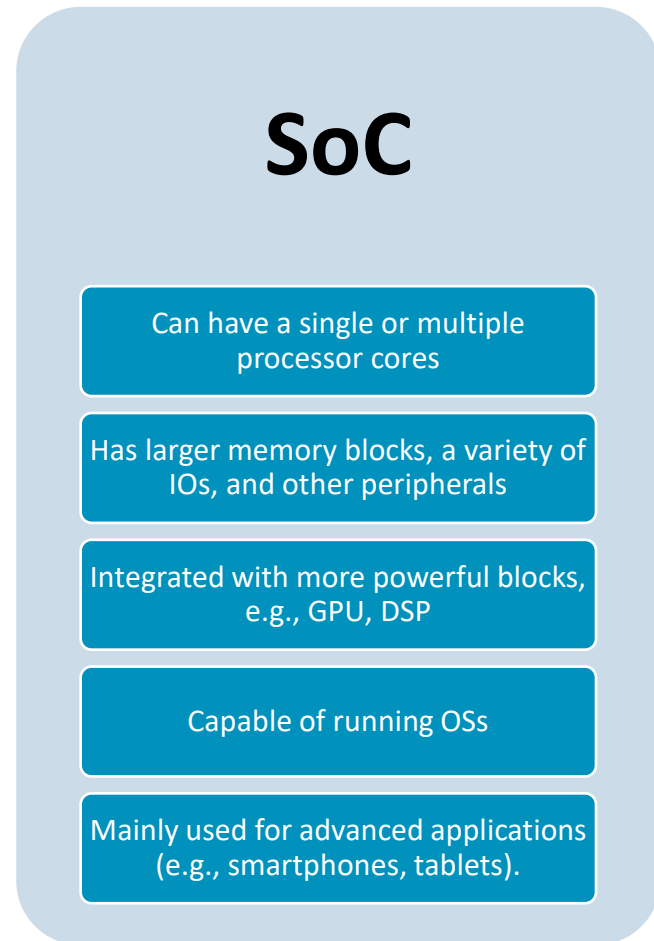
Motherboard of a PC



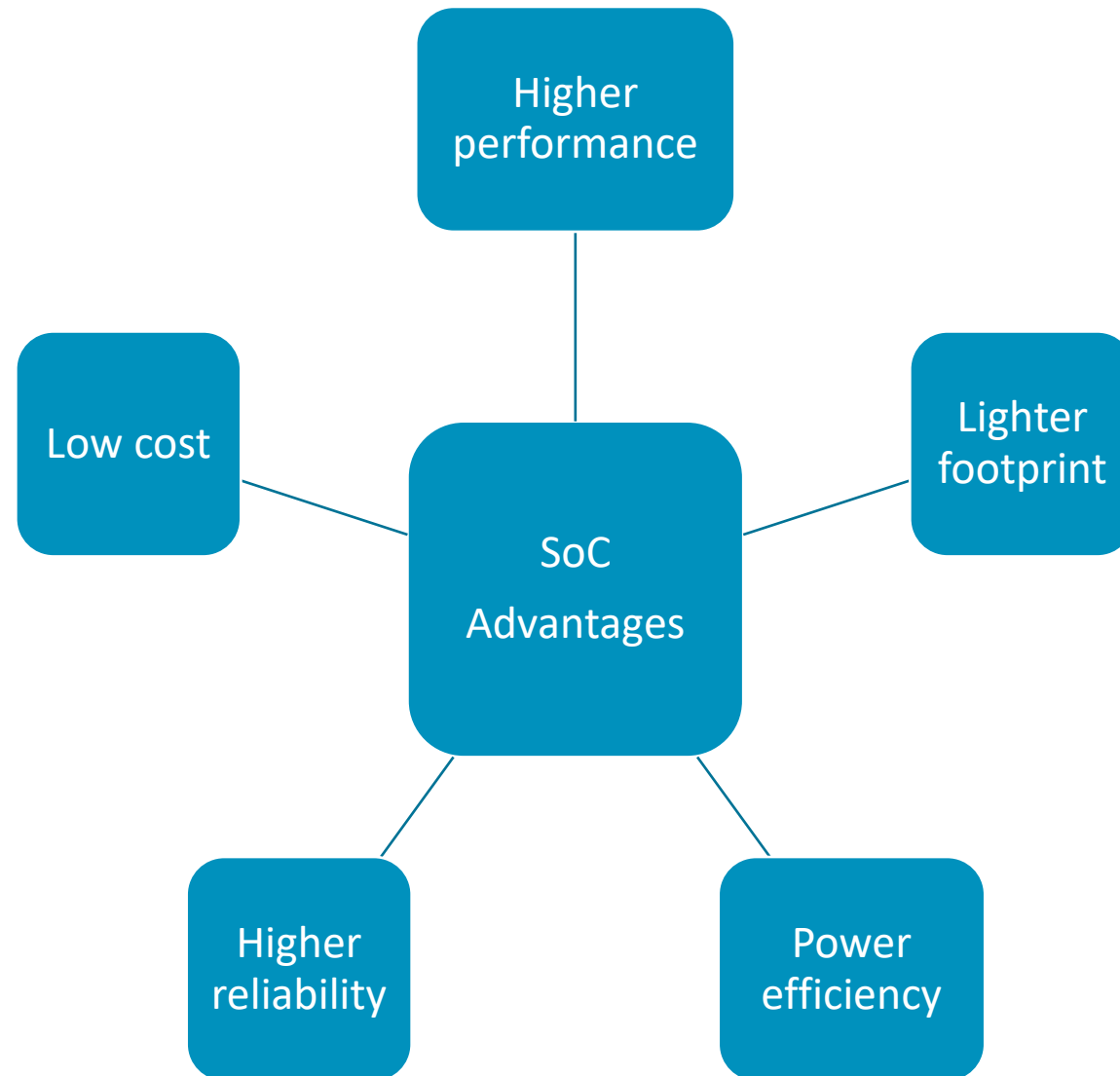
SoC



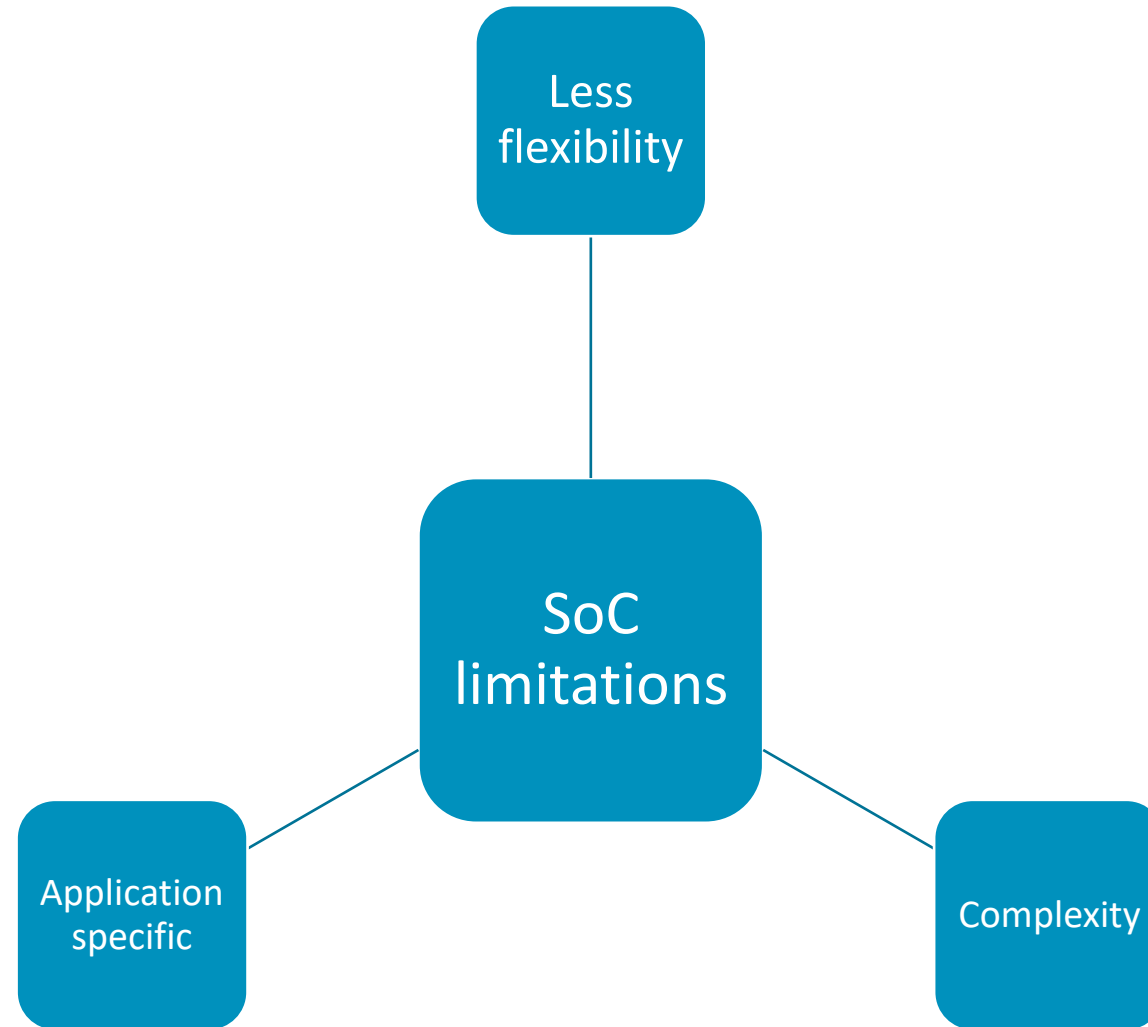
SoC v Microcontroller v Processor



Advantages of SoCs



Limitations of SoCs



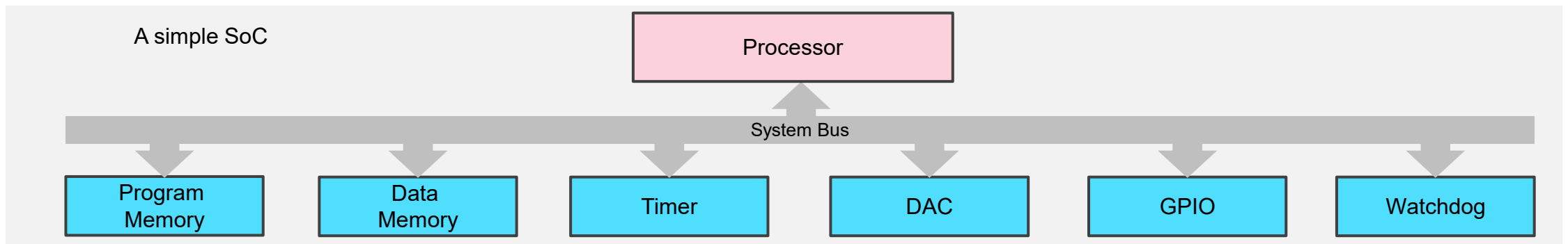
Commercial SoCs

- Benefiting from its power efficiency, SoCs have been widely used in mobile devices, such as smartphones, tablets, and digital cameras.
 - Mainly embedded systems
- A number of SoCs have been developed by a large ecosystem of design companies:
 - Snapdragon by Qualcomm
 - Tegra by Nvidia
 - Apple A-Series
 - Samsung Exynos
 - Raspberry Pi

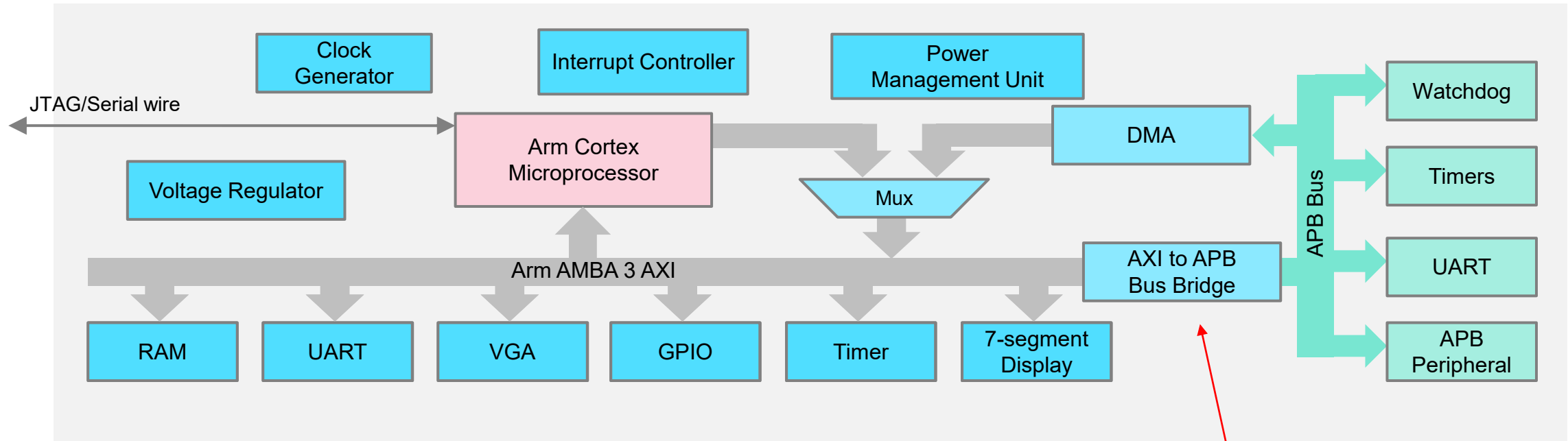
SoC Architecture

What is inside an SoC?

- The basic components of an SoC include:
 - One or more masters/requesters, such as a microprocessor
 - Peripherals, such as accelerators, memory blocks, timers, and external digital/analog interfaces
 - A system bus that connects master and peripherals using a specific bus protocol
- “Architecture” describes how these blocks are connected



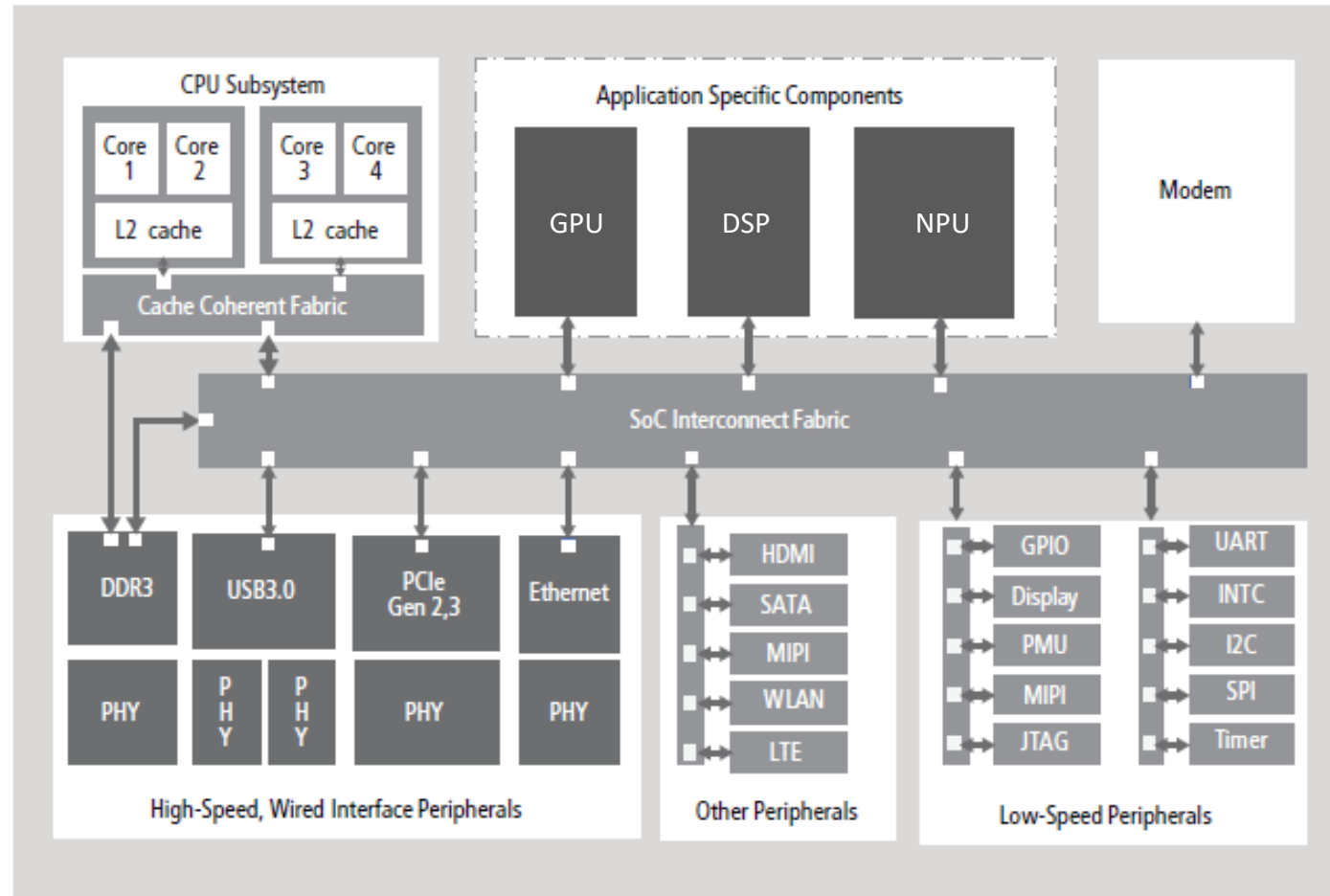
Example Arm-based SoC



Also called gasket
or converter

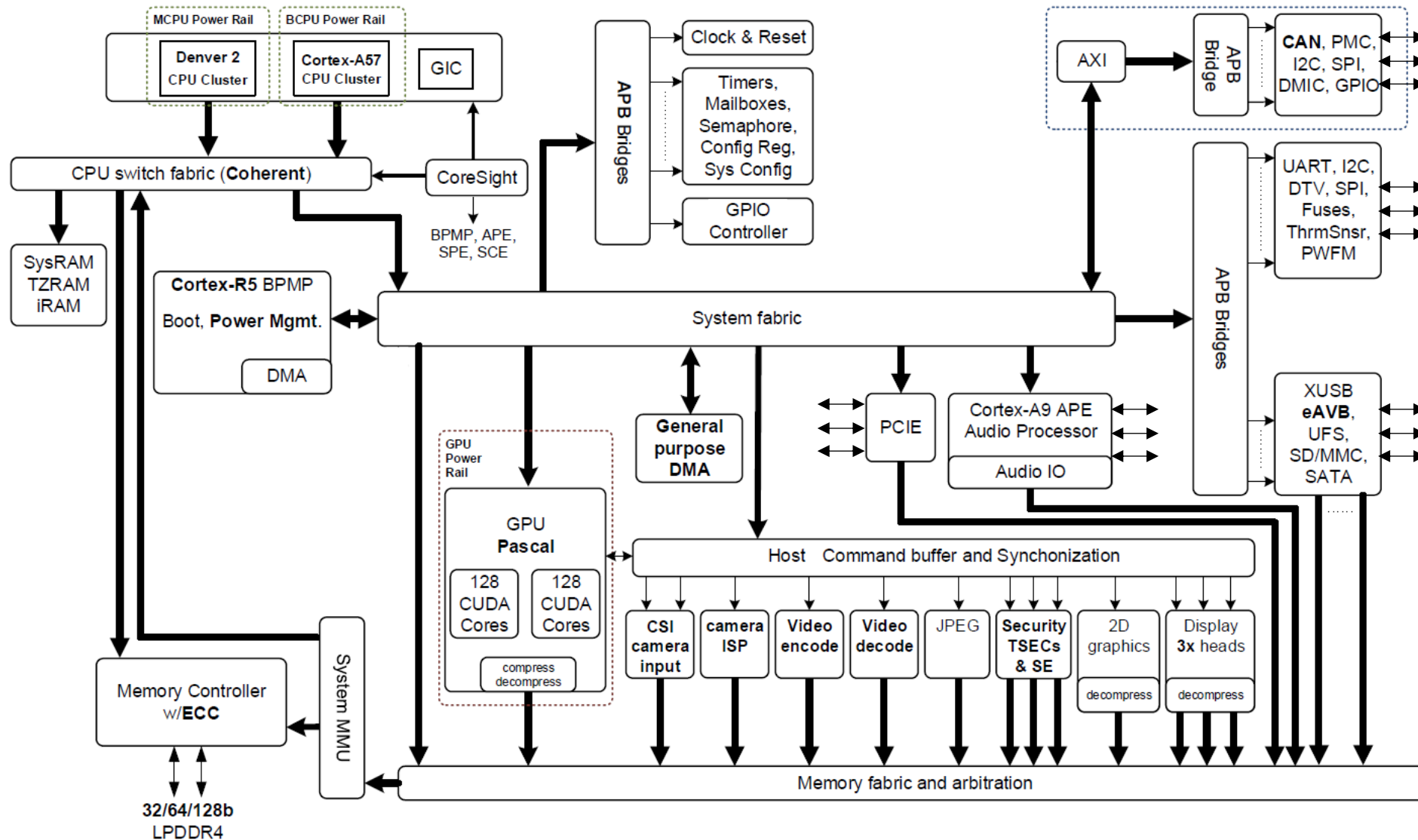
Note the second master/initiator: DMA

Another Example



A more complex SoC from NVIDIA

Locate the I/O pins



CPU subsystem
Multimedia subsystem
Boot subsystem
Network/Backbone
AON subsystem
LSIO subsystem
HSIO subsystem
Memory subsystem

Terminology

- Master, Initiator, Manager
- Slave, Target, Responder, Subordinate

Globals

- Clock and Reset management
- Power management
- Security
- Debug
- Interrupt/Exception management

SoC development is not just integrating some RTL IPs together

System Address Map (SAM)

- SAM provides a structured and organized way for the processor and other system components to access memory and peripherals. It defines how different parts of the chip are laid out in the memory address space.

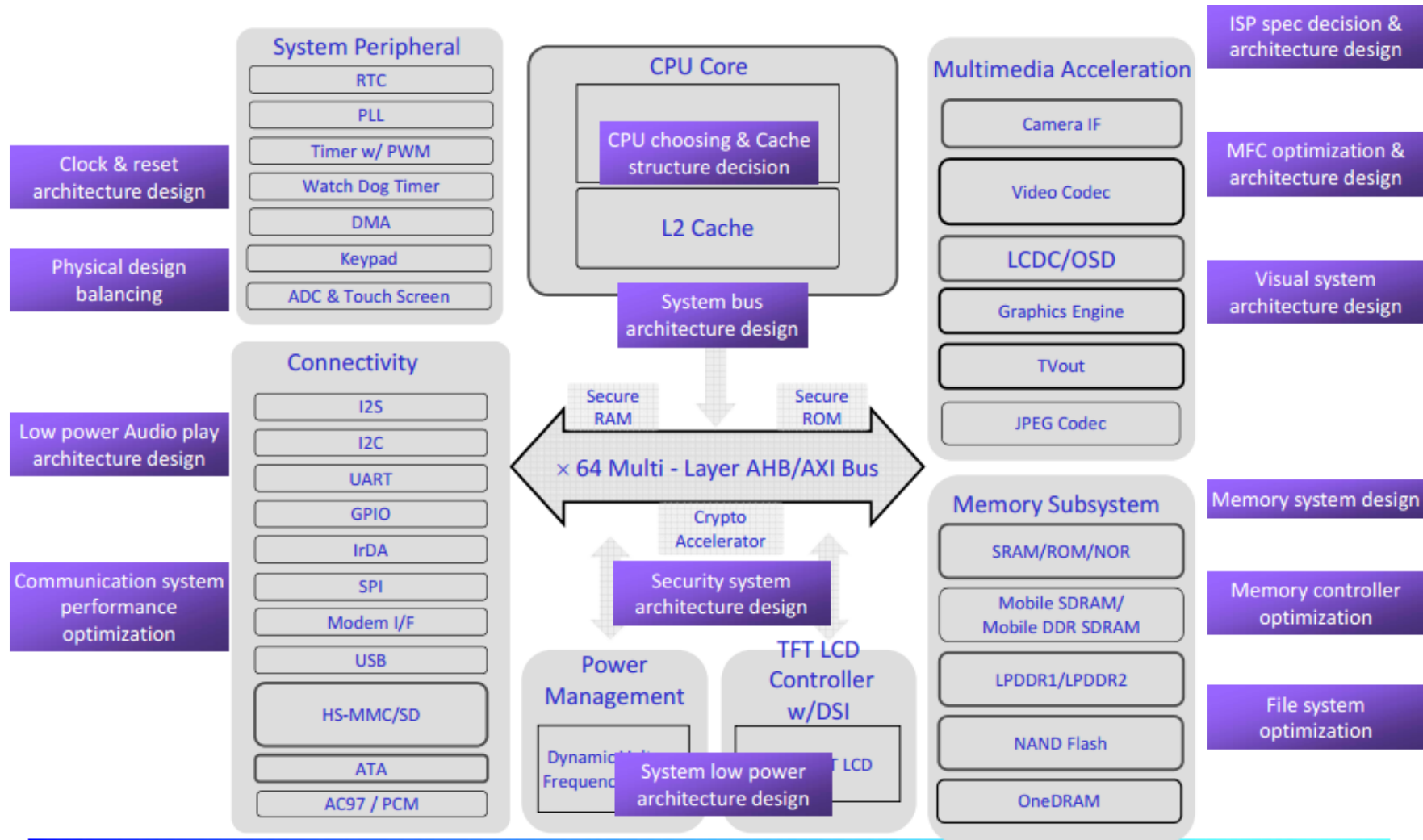
Address	Block/IP
0x0000_0000–0x0FFF_FFFF	Boot ROM
0x2000_0000–0x2FFF_FFFF	SRAM
0x4000_0000–0x4000_0FFF	UART
0x4000_1000–0x4000_1FFF	Timer
0x4000_2000–0x4000_2FFF	GPIO
0x8000_0000–...	DRAM

SoC Architecture Challenges

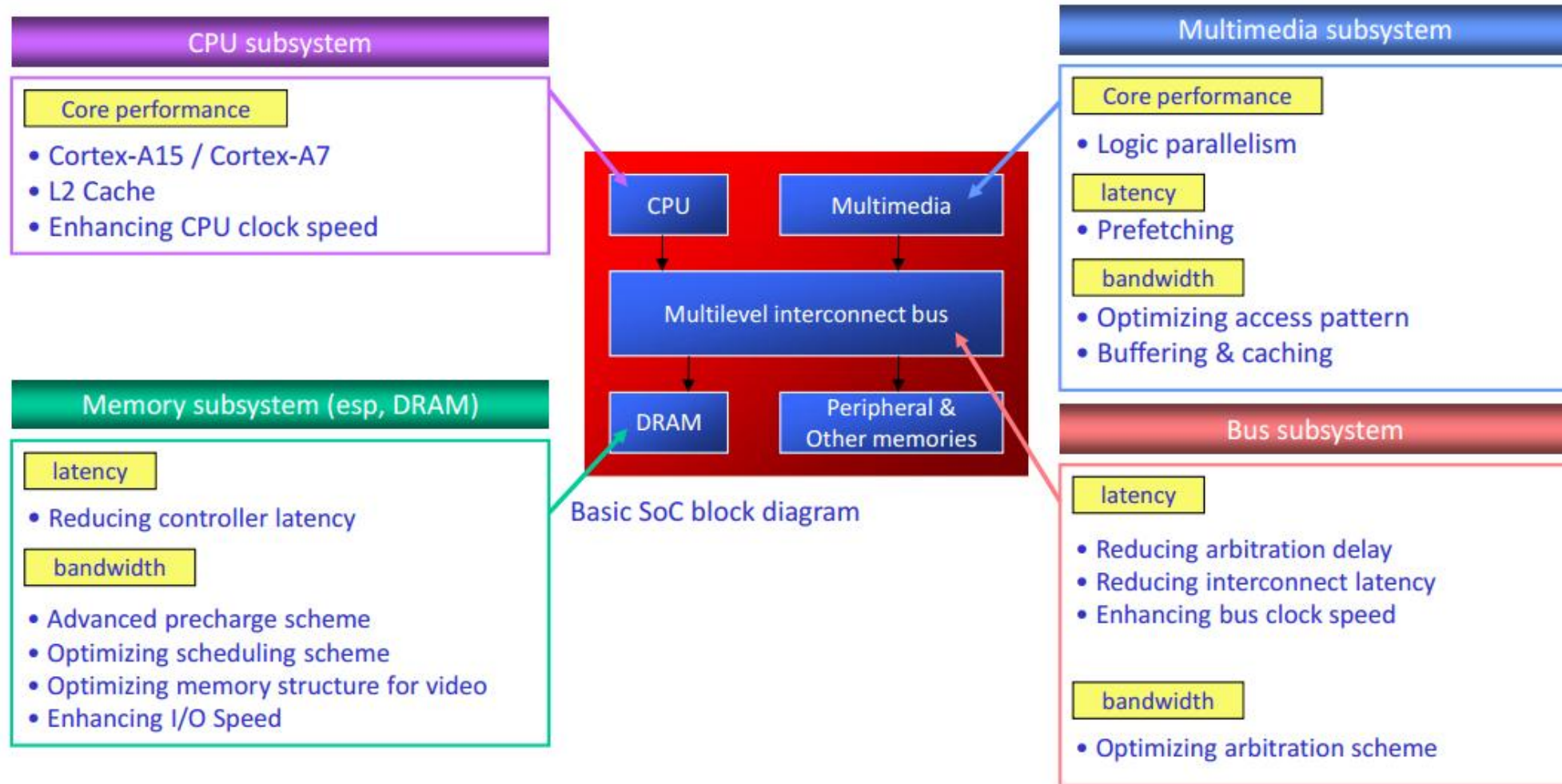
- What compute engines? CPU / GPU / DSP / NPU / accelerators
- What memories? SRAM / caches / DRAM / ROM
- How are they interconnected?
- Who can access what? → address map
- How much bandwidth does each block need?
- How many clock and power domains?
- How do blocks communicate events? → interrupts
- How does data move without the CPU? → DMA
- How does the chip boot, reset, debug, and recover?

Constraints: Performance, Power, Area, Cost, Security

SoC Architecture Challenges



SoC Performance



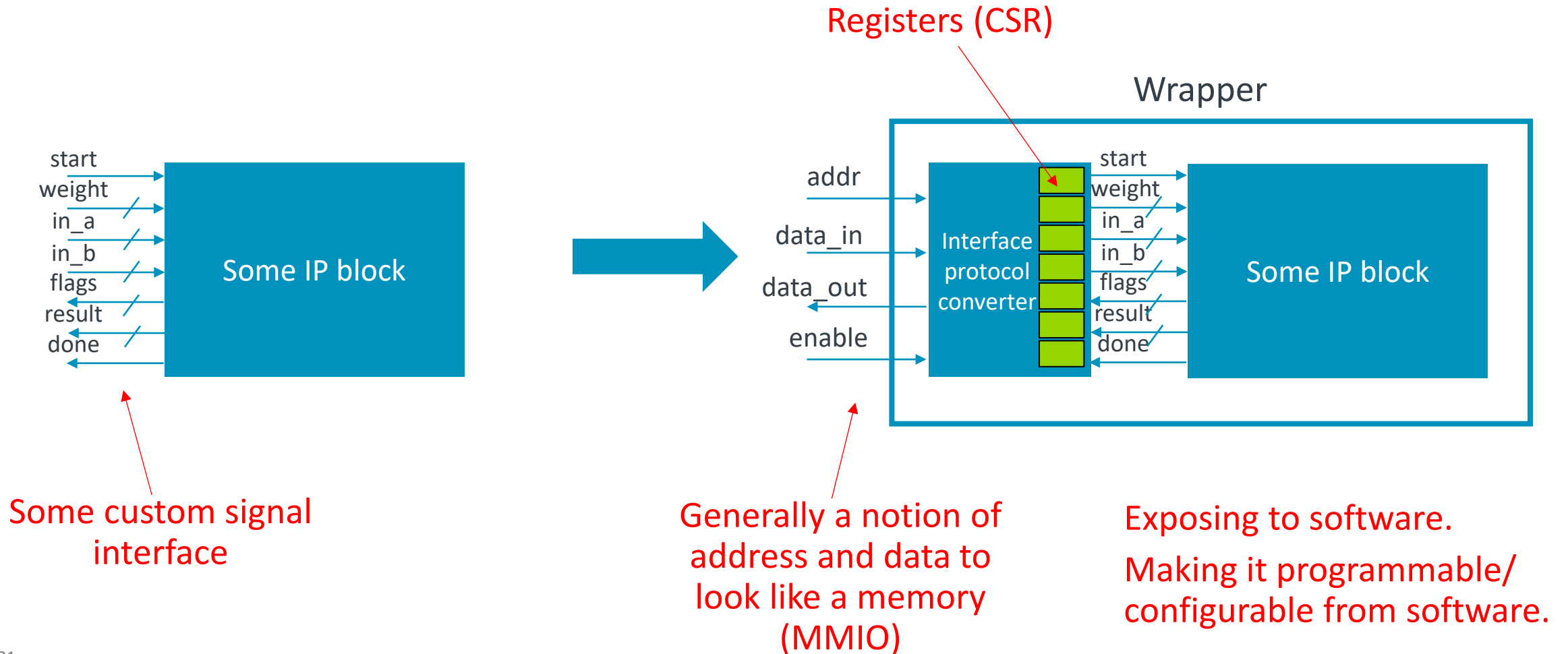
SoC Design

SoC Design Complexity

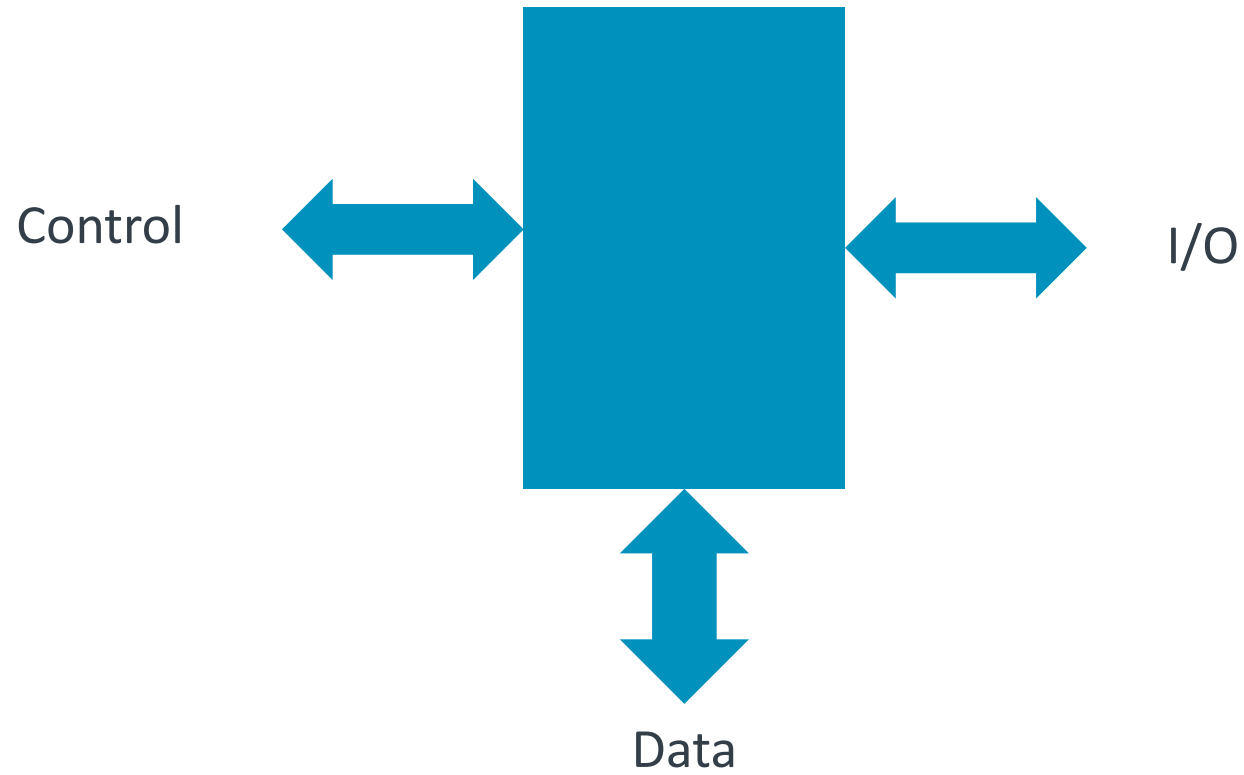
- IP-based design
 - IP = Parameterized components with standard interfaces
 - IPs available in three forms
 - Hard: Black-box in optimized layout form and encrypted simulation model.
 - Firm: Synthesized netlist which can be simulated and changed if needed
 - Soft: Register transfer level (RTL) HDLs; user is responsible for synthesis and layout
- SoC is not just a large ASIC
 - Integrate multiple IPs/sub-systems together (“SoC Integration”)
 - Lot of “glue logic” or “shim logic” in between
 - Cross-domain analysis and optimization
 - Lots of software

Interface Standardization

- IP based design motivates standardization of interfaces



Typical interfaces on an IP block



Life of a “Control” Transaction

- A master/initiator (like the CPU) initiates a transaction
 - Address is usually a MMIO address
 - “I’d like to configure some bit in the UART engine”
- The transaction comes out on the CPU’s port to the control interconnect/backbone
 - Typically, low bandwidth interconnect
- The interconnect has the address map. It decodes where the transaction needs to go.
- The transaction is routed to the destination
 - There may be multiple levels (bridges) to get to the destination
- Transaction is executed by the peripheral
 - May be a CSR or a memory inside a peripheral
- Completion/response is sent back to the initiator/master

Consumes time: CPU → interconnect → arbitration → bridge → clock-domain crossing → peripheral → response → interconnect → CPU.

Life of a “Data” Transaction

- A master/initiator (like the CPU or an accelerator) initiates a transaction
 - “I’d like to read data from address DEF”
 - Address is for a memory location (rather than a peripheral register)
- The transaction comes out on the CPU’s/Accelerator’s port to the data interconnect/backbone
 - Typically high bandwidth interconnect
- The interconnect arbitrates
 - Multiple initiators may simultaneously request data from the memory
- Transaction reaches the memory subsystem
 - Memory controller schedules the request to the memory
- Data is transferred (read/write particular addresses) as one or more bursts/beats
- Completion/Response is sent back to the master/requester

Control vs. Data Transaction

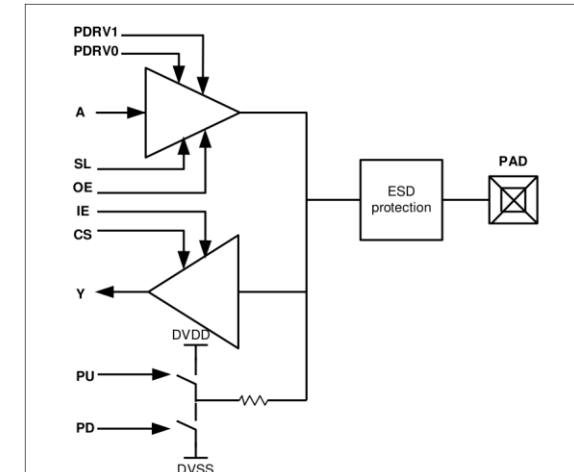
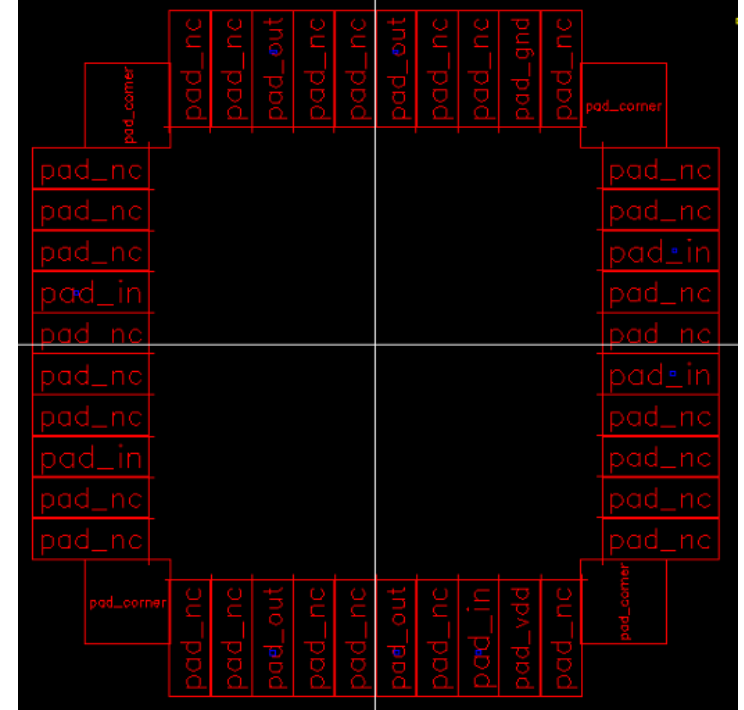
Control transaction	Data transaction
Usually small	Often large/bursty
Register/MMIO access	Memory access
Latency-oriented	Bandwidth/throughput-oriented
CPU often initiator	CPU, DMA, GPU, accelerator
Simple peripheral network	High-performance NoC/interconnect
e.g. “Start accelerator”	e.g. “Read 4 KB of input data”

Domains

- Clock domains
 - CDC, synchronizers, FIFOs
- Reset domains
 - Reset sequencing
- Power domains
 - Isolation cells
 - Always-On domain

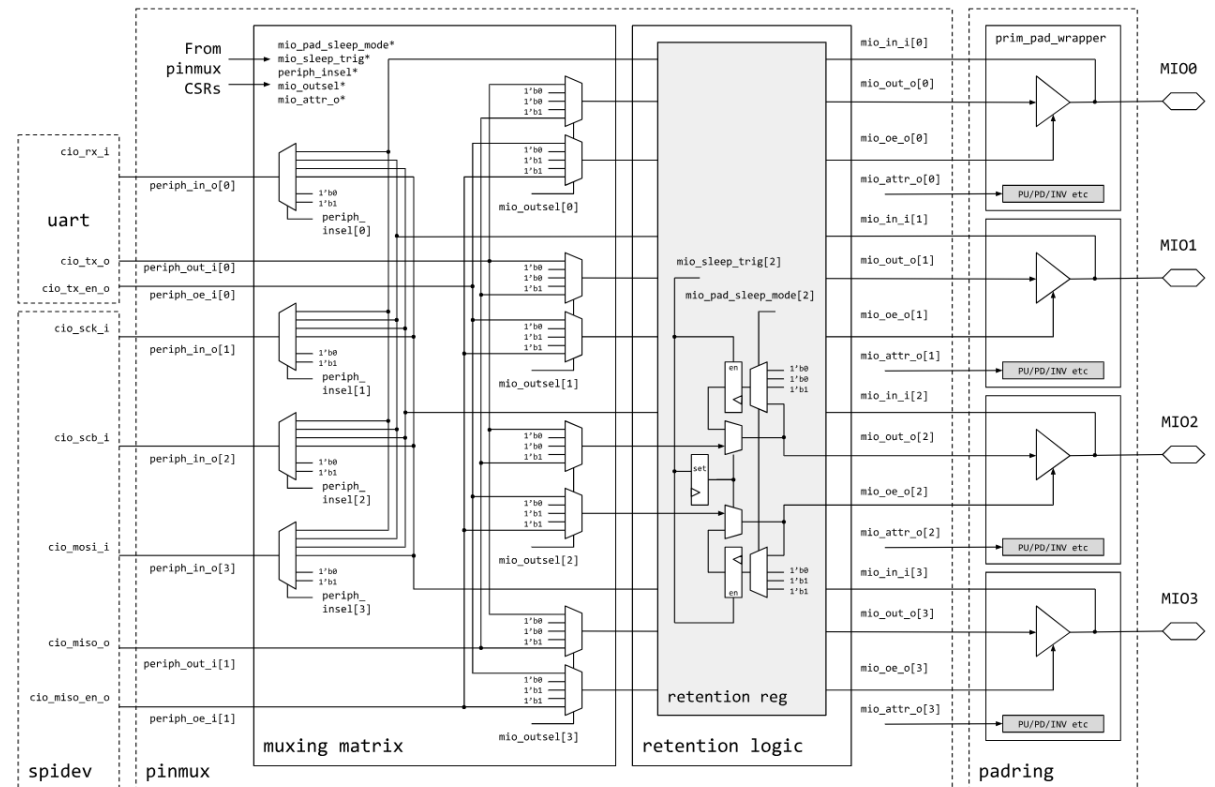
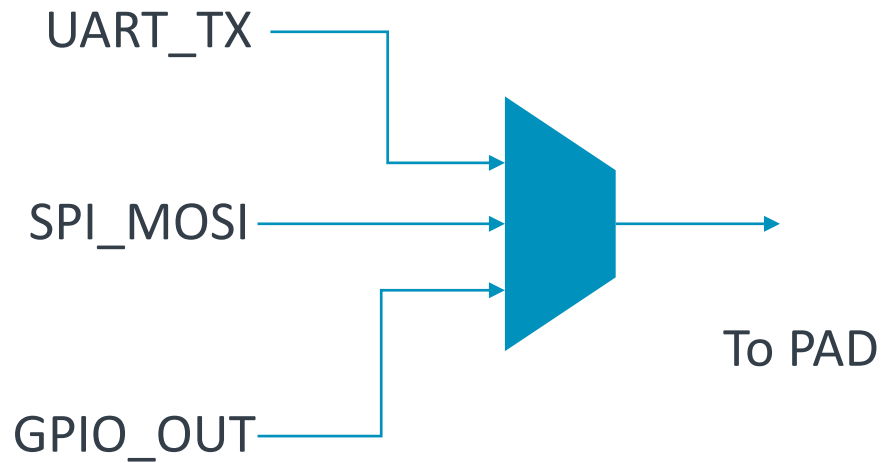
Padring

- Can't connect RTL wires directly to the outside world
- I/O pads are input/output circuits used as intermediate structures connecting internal signals from the core of the SoC to the external pins of the chip package
- Typically organized into a ring at the periphery of the chip
- Multiple power pads are often used for providing voltage/power to the chip
- I/O pads usually consist of
 - Level shifters and buffers
 - ESD protection circuitry
 - Configurable slew rate
 - Pullup/Pulldown circuitry
 - Output drive enable



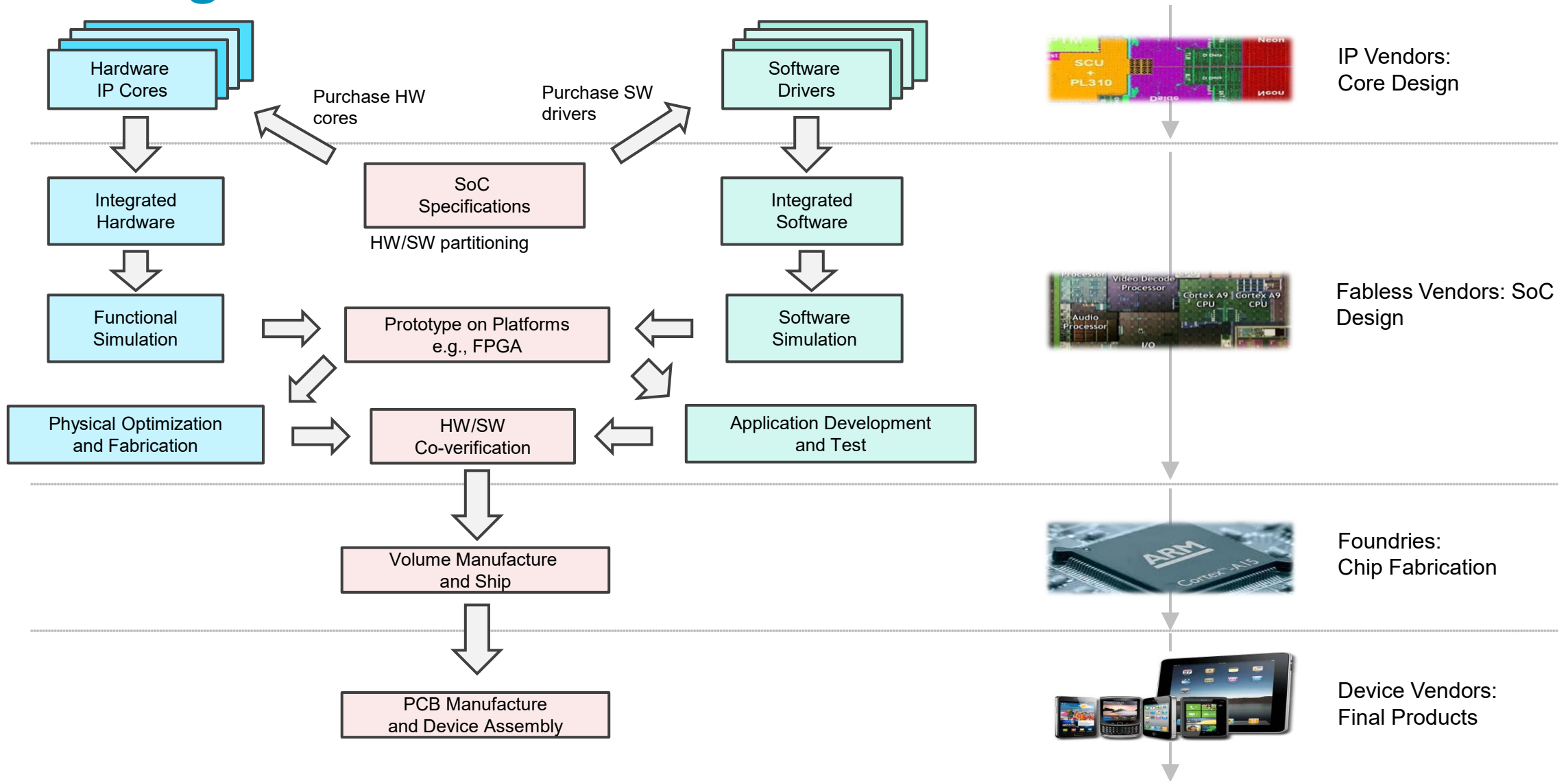
Pinmux

- An SoC may contain hundreds of internal signals (from various blocks) but cannot afford a separate physical pin/pad for every signal
- Pinmux lets multiple blocks share physical pins/pads with configuration registers determining the mapping

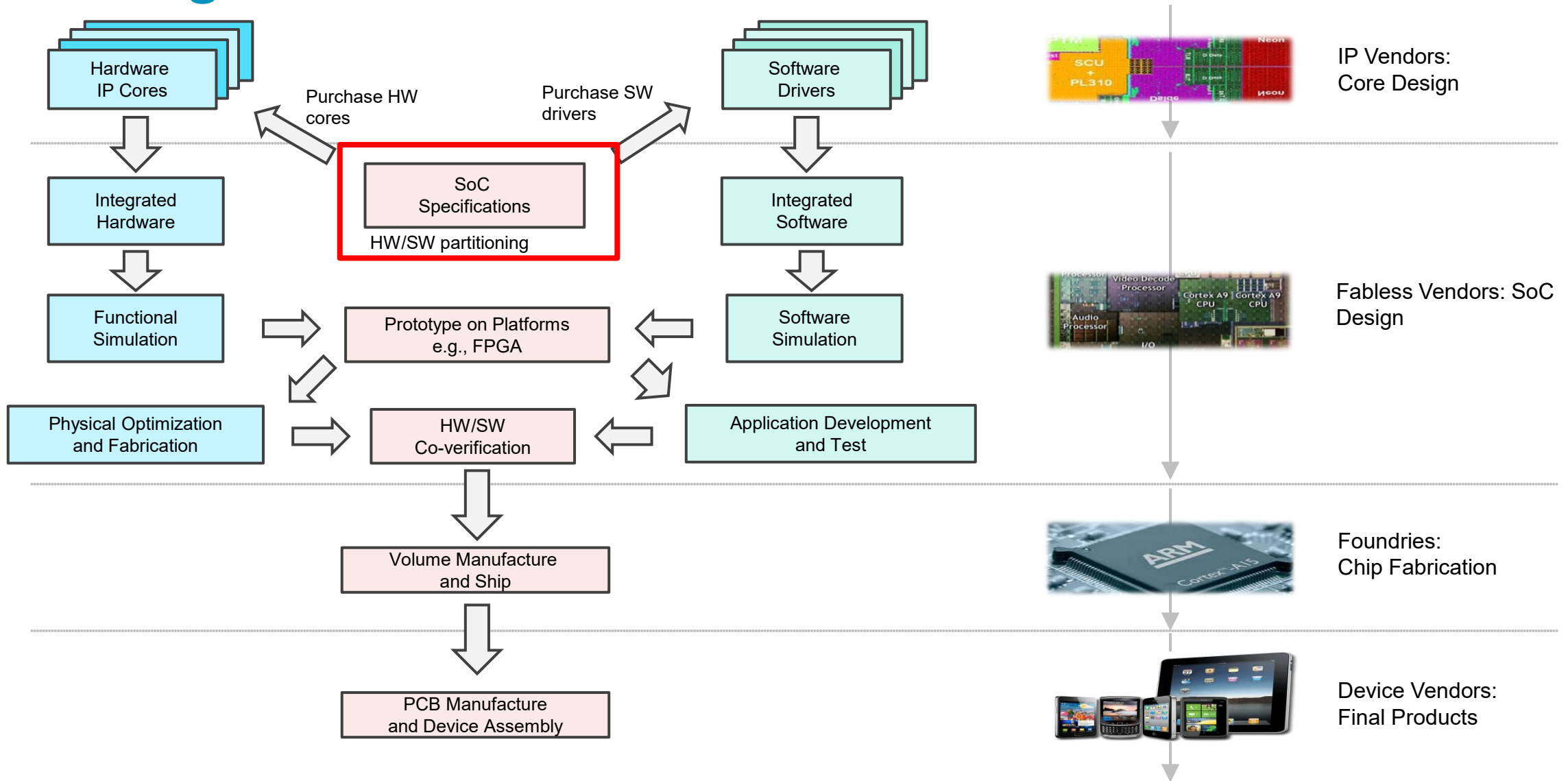


SoC Design Flow

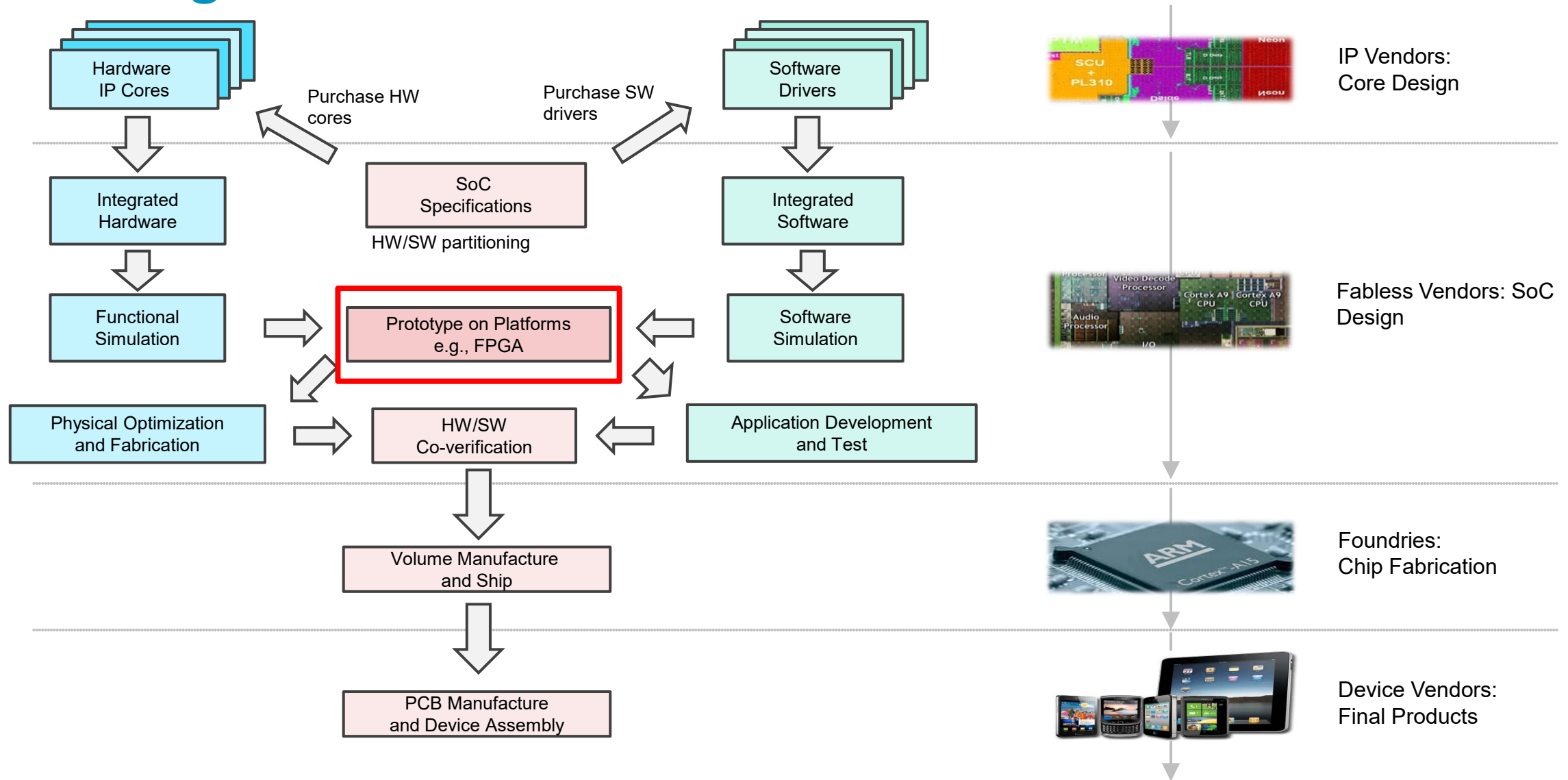
SoC Design Flow



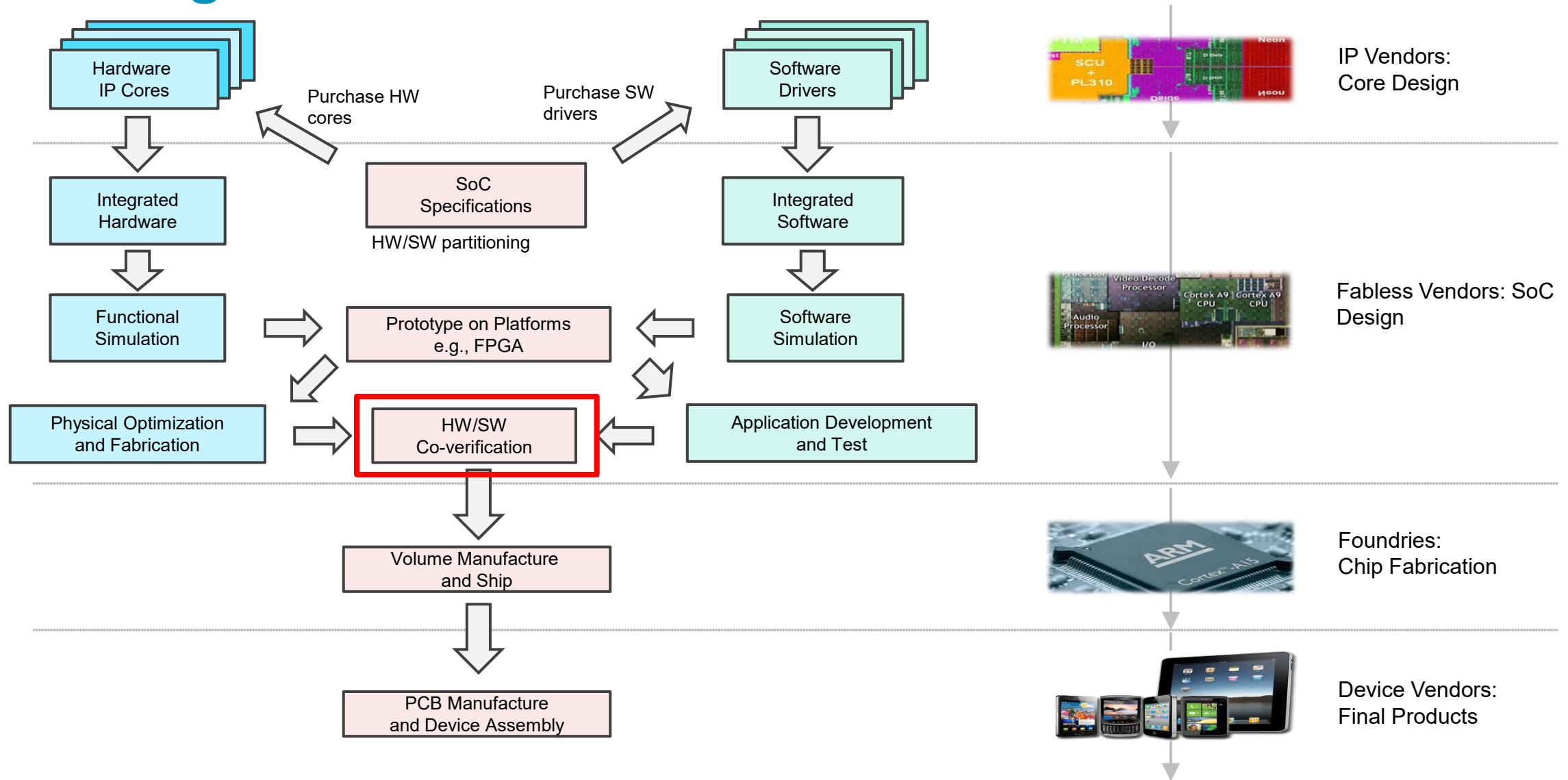
SoC Design Flow



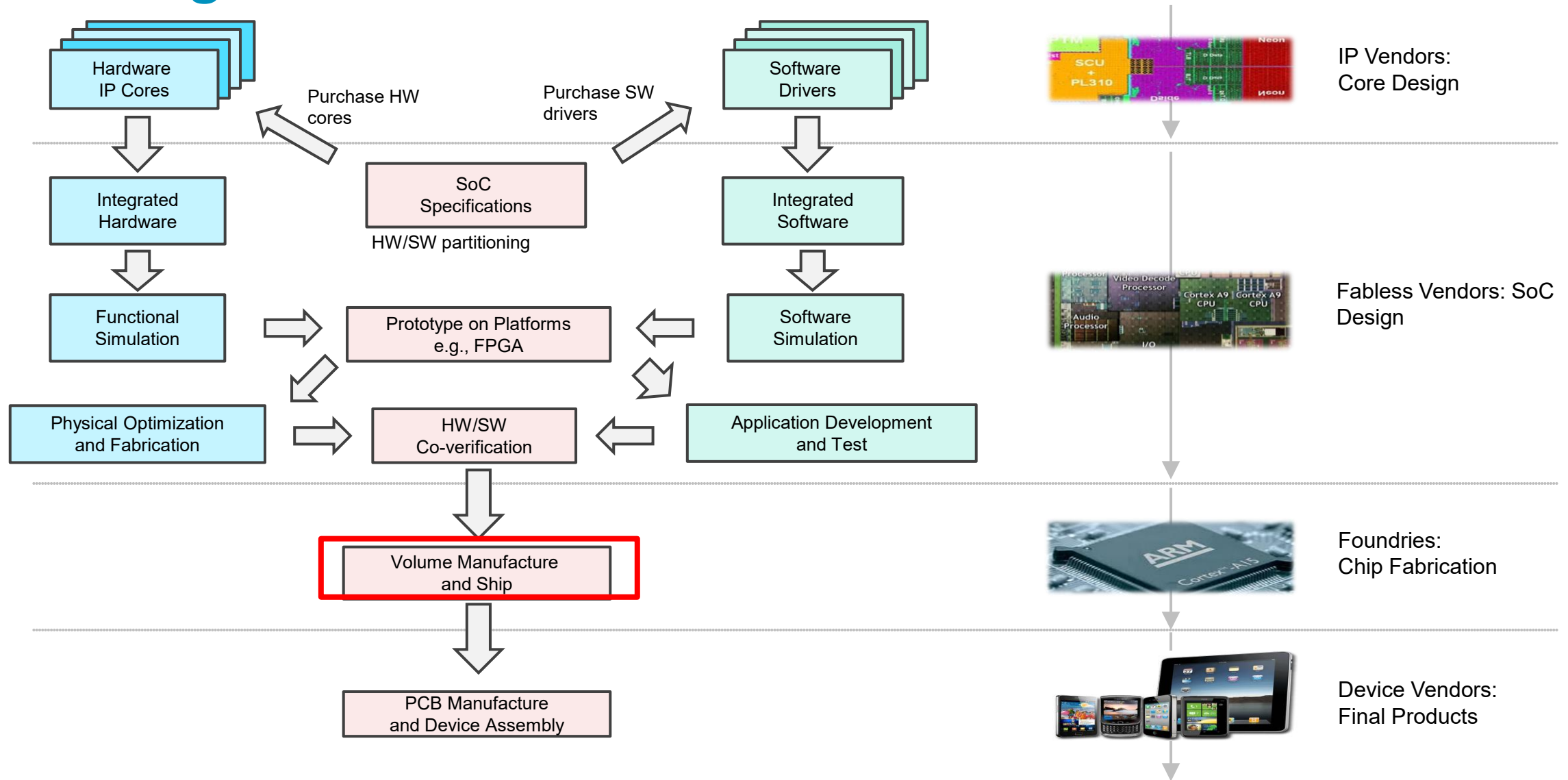
SoC Design Flow



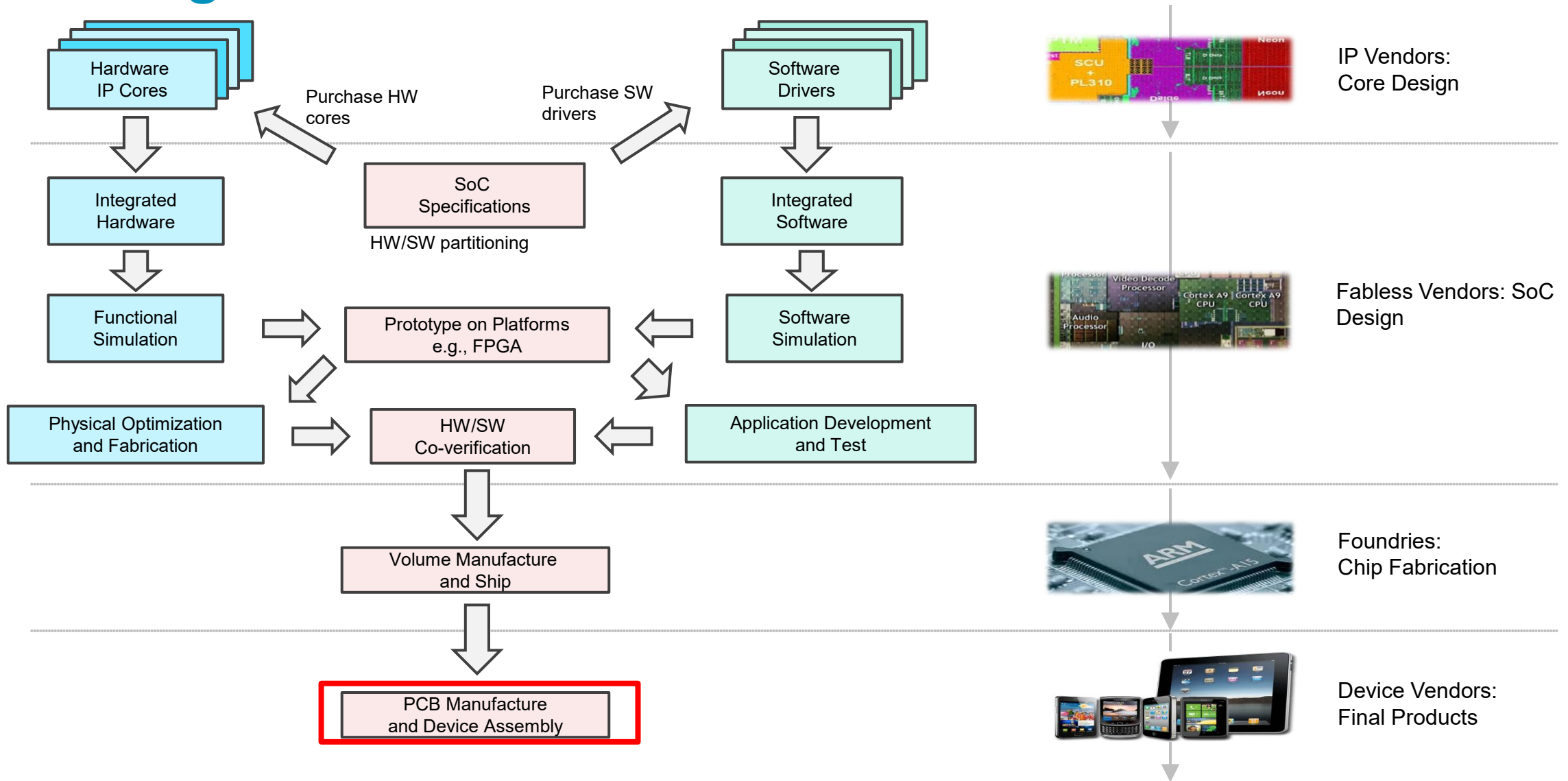
SoC Design Flow



SoC Design Flow



SoC Design Flow



SoC Interfaces

On-Chip Interfaces

- Protocols
 - AMBA APB, AXI
- Interconnect styles
 - Shared bus
 - Only one/few transactions at once.
 - Crossbar
 - Multiple simultaneous paths
 - NoC
 - Packets, routers, links

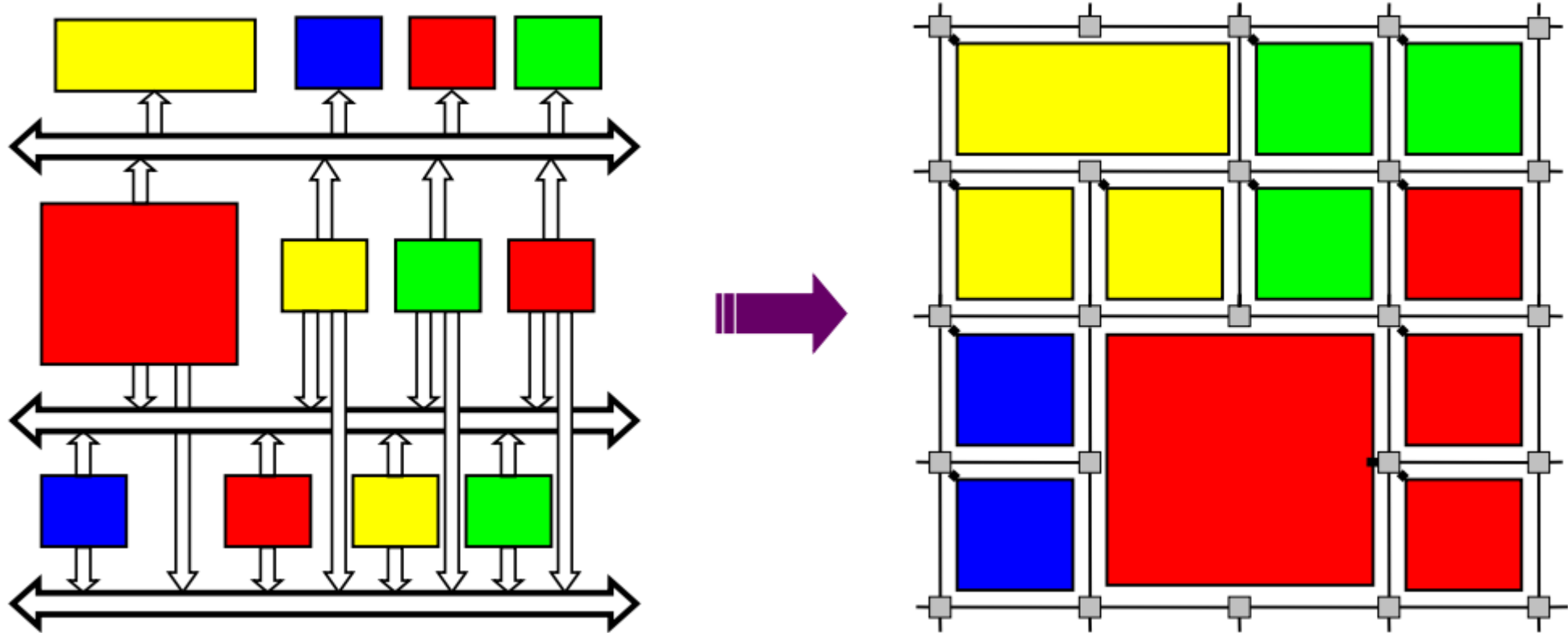


The topic of next lecture.

Sometimes the words NoC and Interconnect are used interchangeably.

Buses to Networks

- Architectural paradigm shift: Replace wire spaghetti by network
- Usage paradigm shift: Pack everything in packets



AMBA (Advanced Microcontroller Bus Architecture)

Key AMBA specifications

		AMBA	AMBA 2	AMBA 3	AMBA 4	AMBA 5
CHI Coherent Hub Interface	A credited coherency protocol Layered architecture for scalability					CHI
ACE AXI Coherency Extensions	A superset of AXI — system-wide coherency across multicore clusters				ACE +Lite	ACE5 +Lite
AXI Adv. eXtensible Interface	AXI supports separate A/D phases, bursts, multiple outstandings addresses, and OoO responses			AXI3	AXI4 +Lite, +Stream	AXI5
AHB Adv. High-performance Bus	AHB supports 64 and 128 bit multi-manager AHB-Lite is for single managers		AHB	AHB +Lite		AHB5 +Lite
APB Advanced Peripheral Bus	System bus for low bandwidth peripherals	APB	APB2	APB3	APB4	

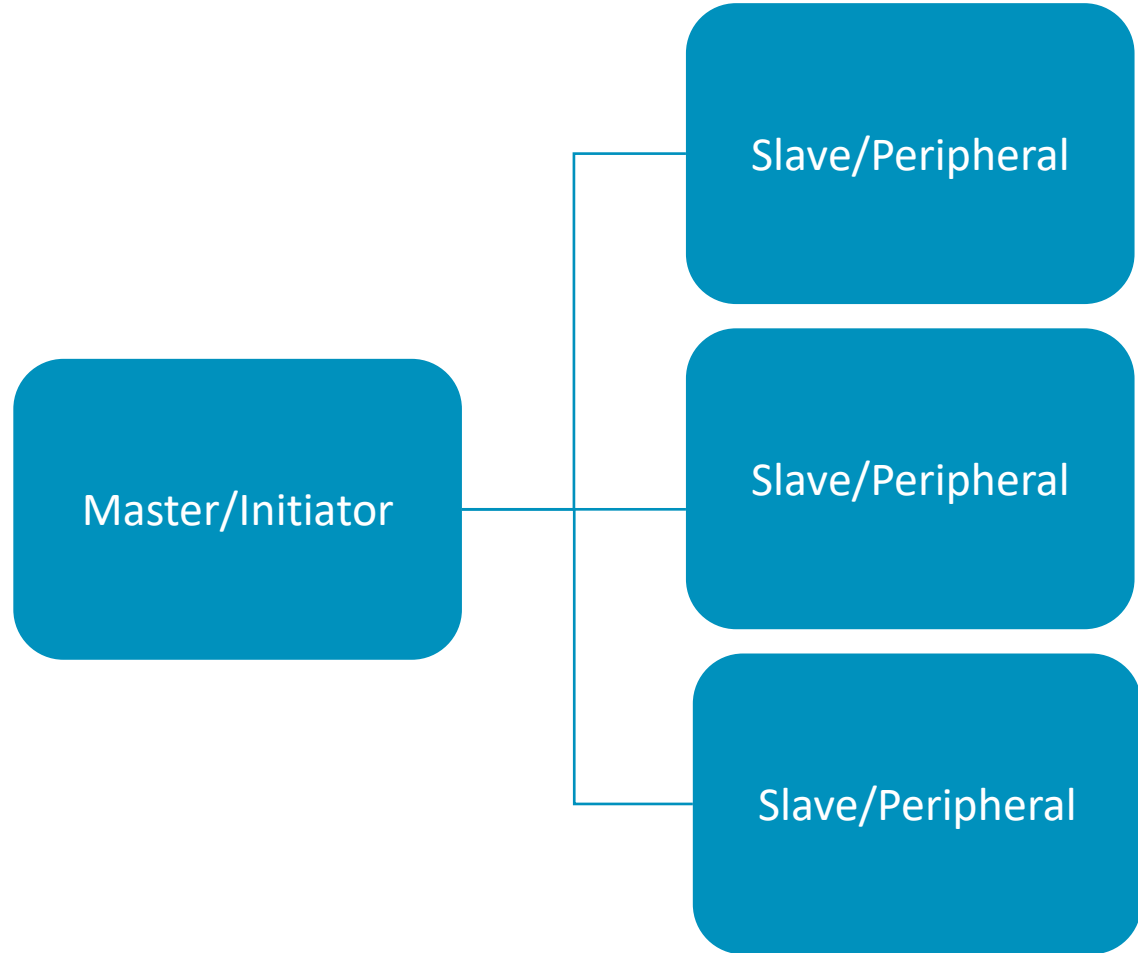
What does the interconnect need to optimize?

- Peripheral register access (“Control”)
 - CPU → UART
 - 4 bytes, infrequent → **latency + simplicity matter**
- Accelerator (“Data”)
 - NPU → DRAM
 - 64 MB tensor → **bandwidth matters**

APB: optimize simplicity/area/power
AXI: optimize throughput/concurrency

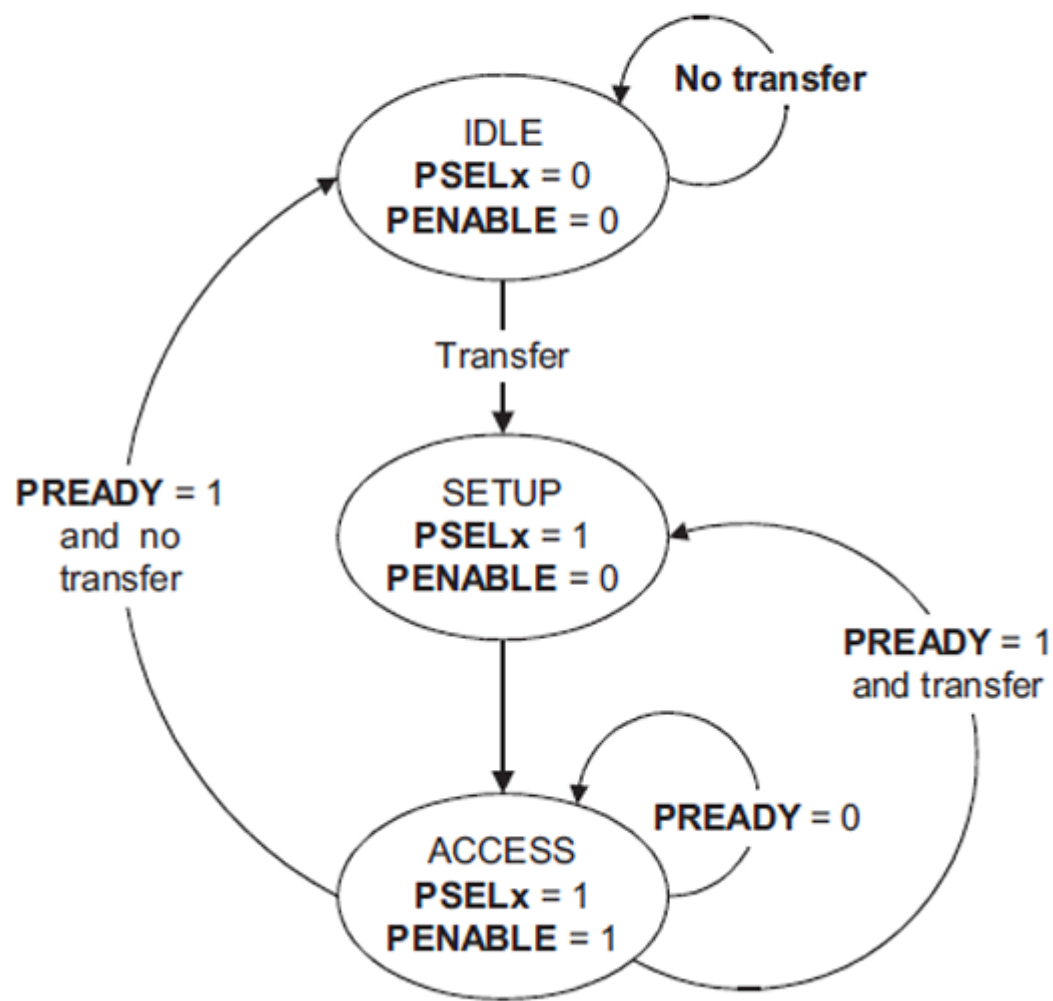
APB (Advanced Peripheral Bus)

- Low-cost
- Low-power
- Low-complexity
- Low-bandwidth
- Non-pipelined
- Ideal for peripherals



APB bus state machine

- **IDLE**
 - Default APB state
- **SETUP**
 - When transfer required
 - **PSELx** is asserted
 - Only one cycle
- **ACCESS**
 - **PENABLE** is asserted
 - Addr, write, select, and write data remain stable
 - Stay if **PREADY** = L
 - Goto IDLE if **PREADY** = H and no more data
 - Goto SETUP if **PREADY** = H and more data pending

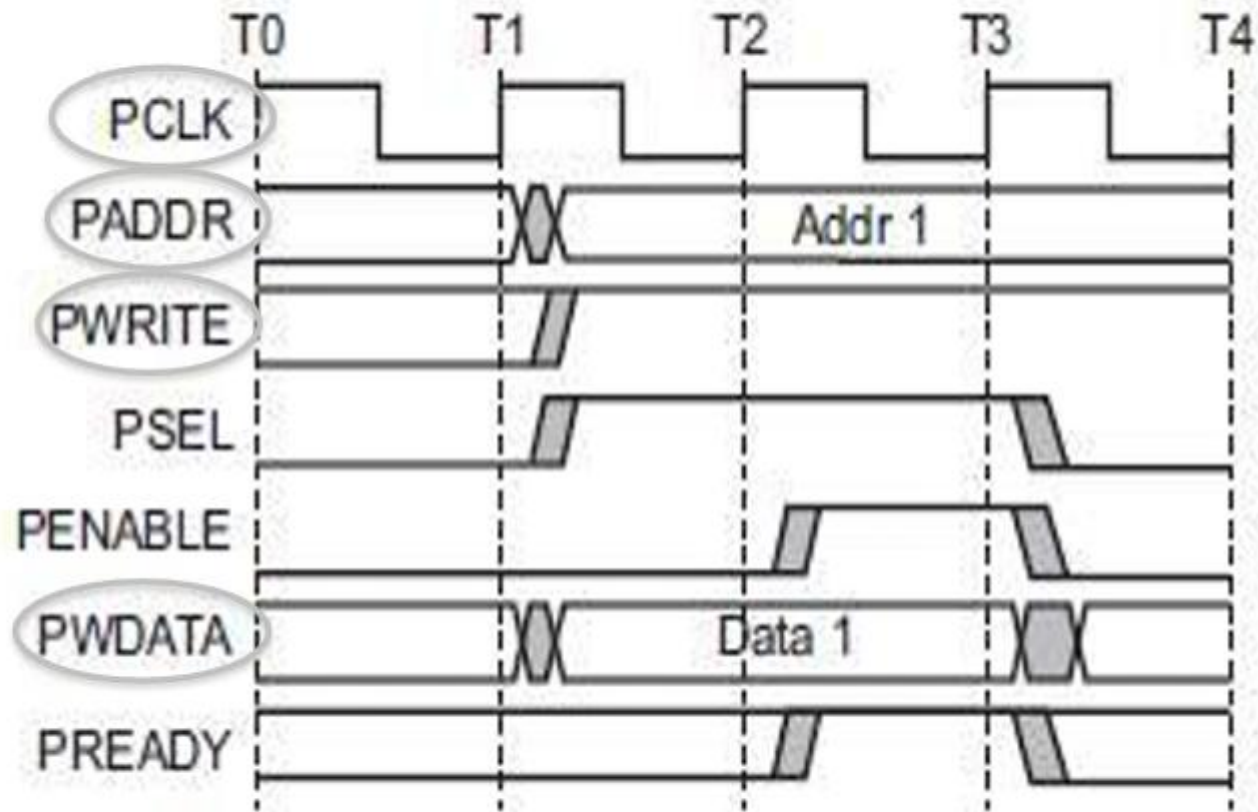


APB Signals

- PCLK: bus clock source (rising-edge triggered)
- PRESETn: bus (and typically system) reset signal (active low)
- PADDR: APB address bus (can be **up to 32-bits** wide)
- PSELx: select line for each slave device
- PENABLE: indicates the 2nd and subsequent cycles of an APB xfer
- PWRITE: indicates transfer direction (Write=H, Read=L)
- PWDATA: write data bus (can be **up to 32-bits** wide)
- PREADY: used to extend a transfer
- PRDATA: read data bus (can be **up to 32-bits wide**)
- PSLVERR: indicates a transfer error
(OKAY=L, ERROR=H)

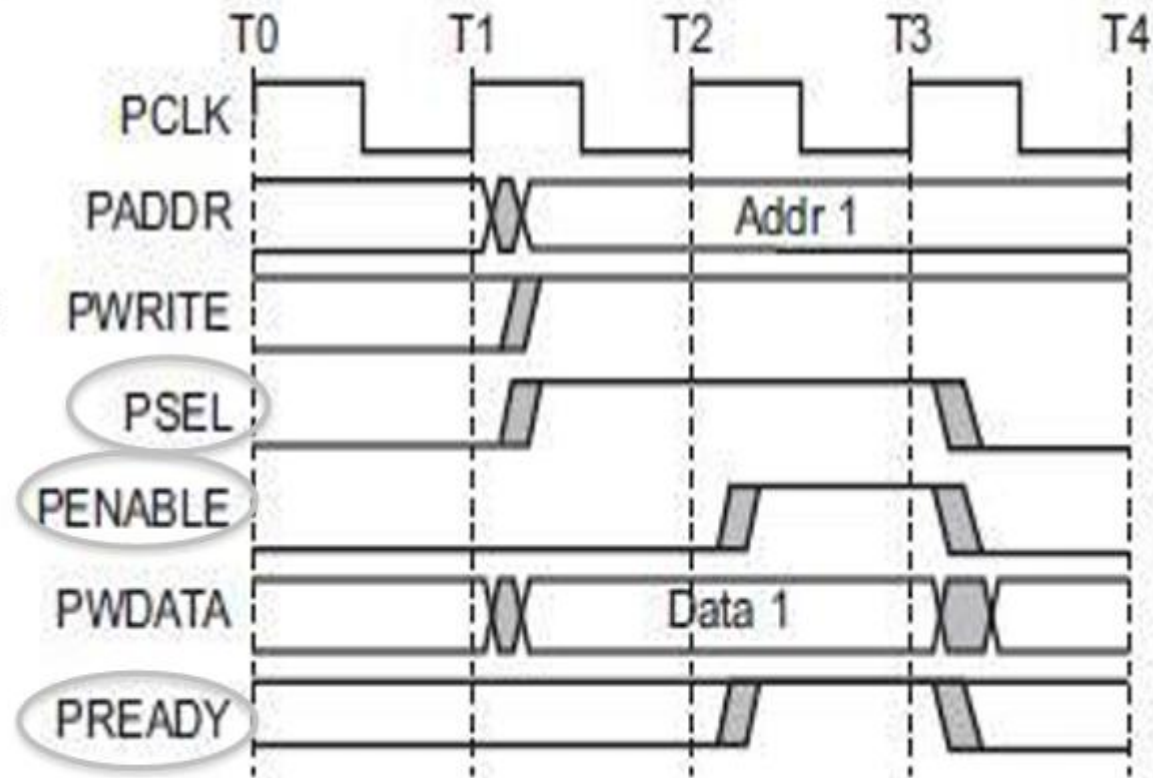
APB bus Signals

- PCLK
Clock
- PADDR
Address on bus
- PWRITE
1=Write,
0=Read
- PWDATA
* Data written to
the I/O device.
* Supplied by
the bus
master/processor.



APB bus signals

- PSEL
 - Asserted if the current bus transaction is targeted to ***this*** device
- PENABLE
 - High during entire transaction *other than* the first cycle.
- PREADY
 - Driven by target. Similar to our #ACK. Indicates if the target is ***ready*** to do transaction.



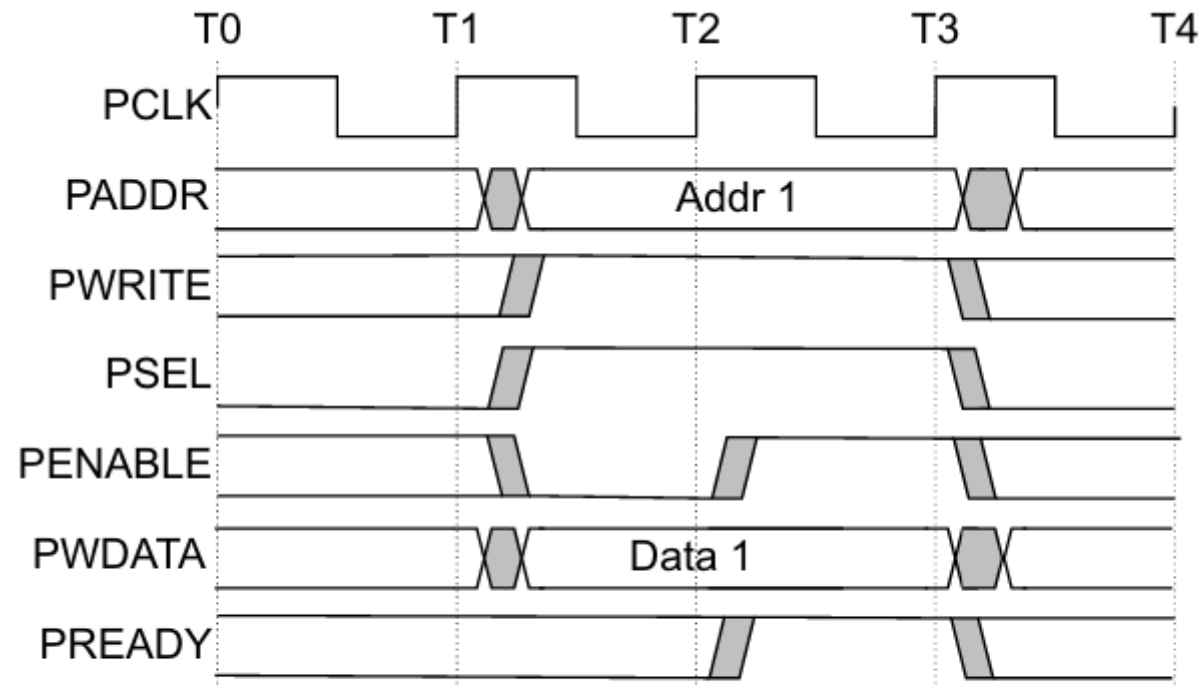


Figure 3-1 Write transfer with no wait states

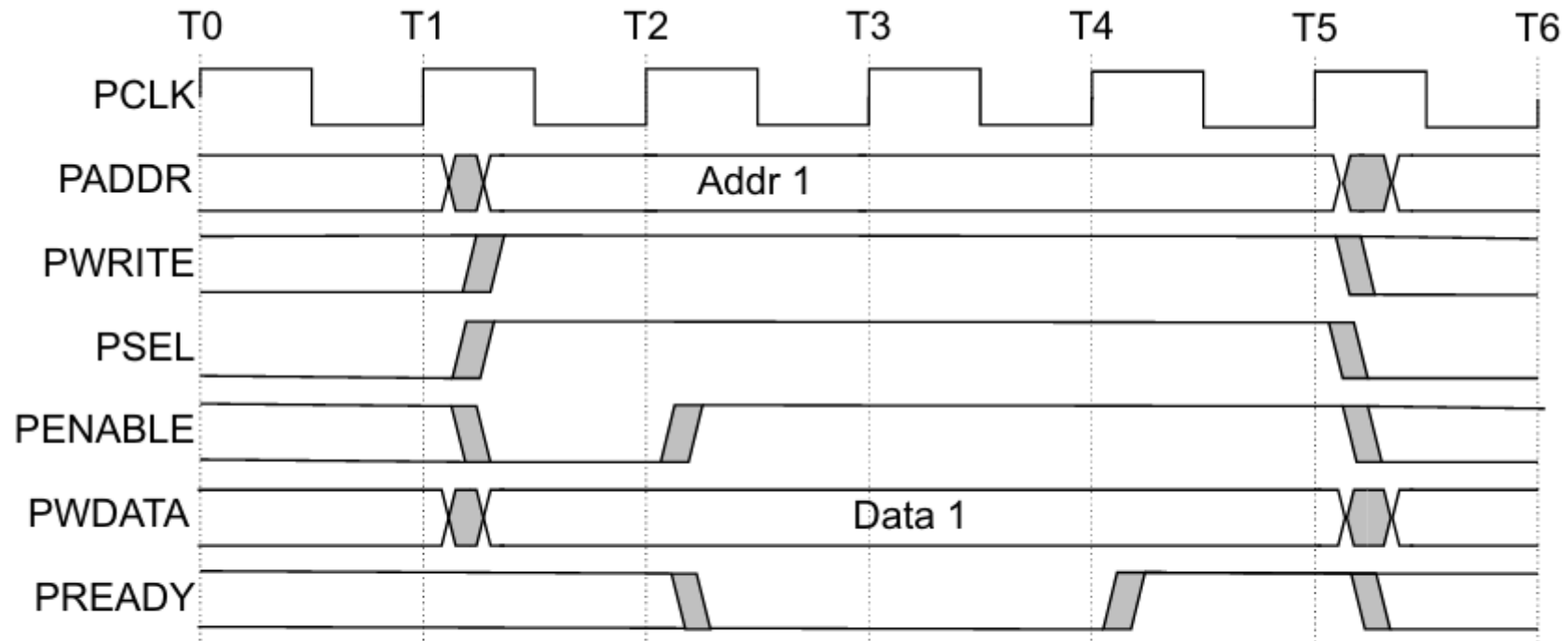


Figure 3-2 Write transfer with wait states

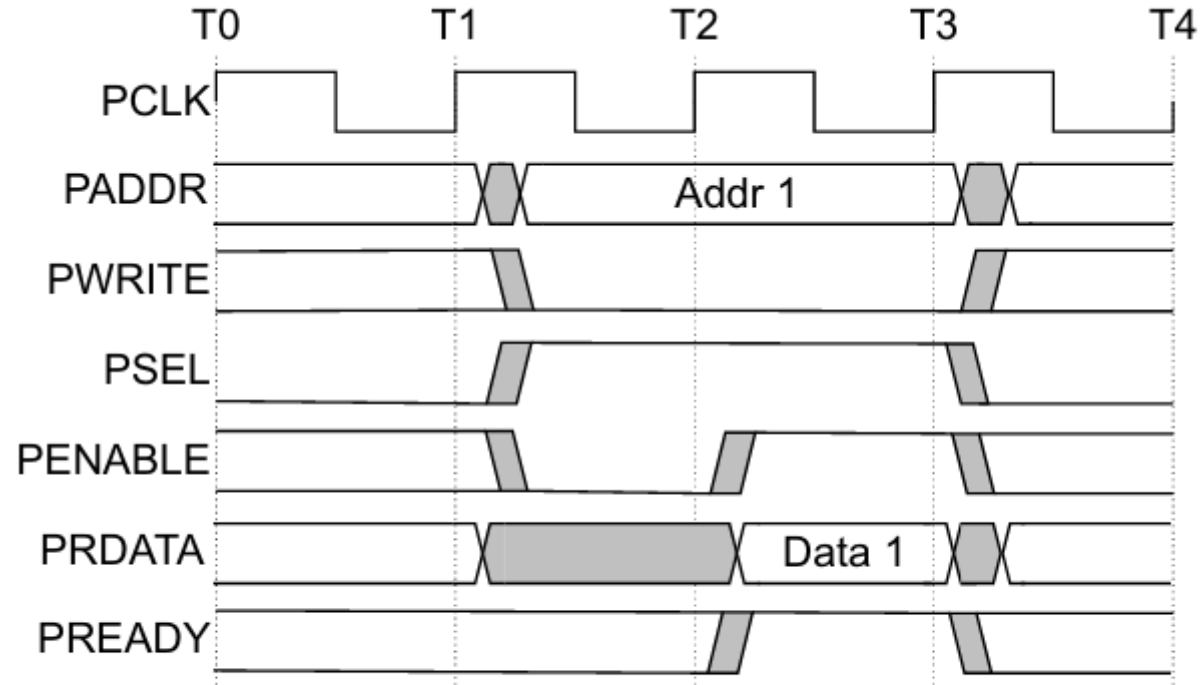


Figure 3-4 Read transfer with no wait states

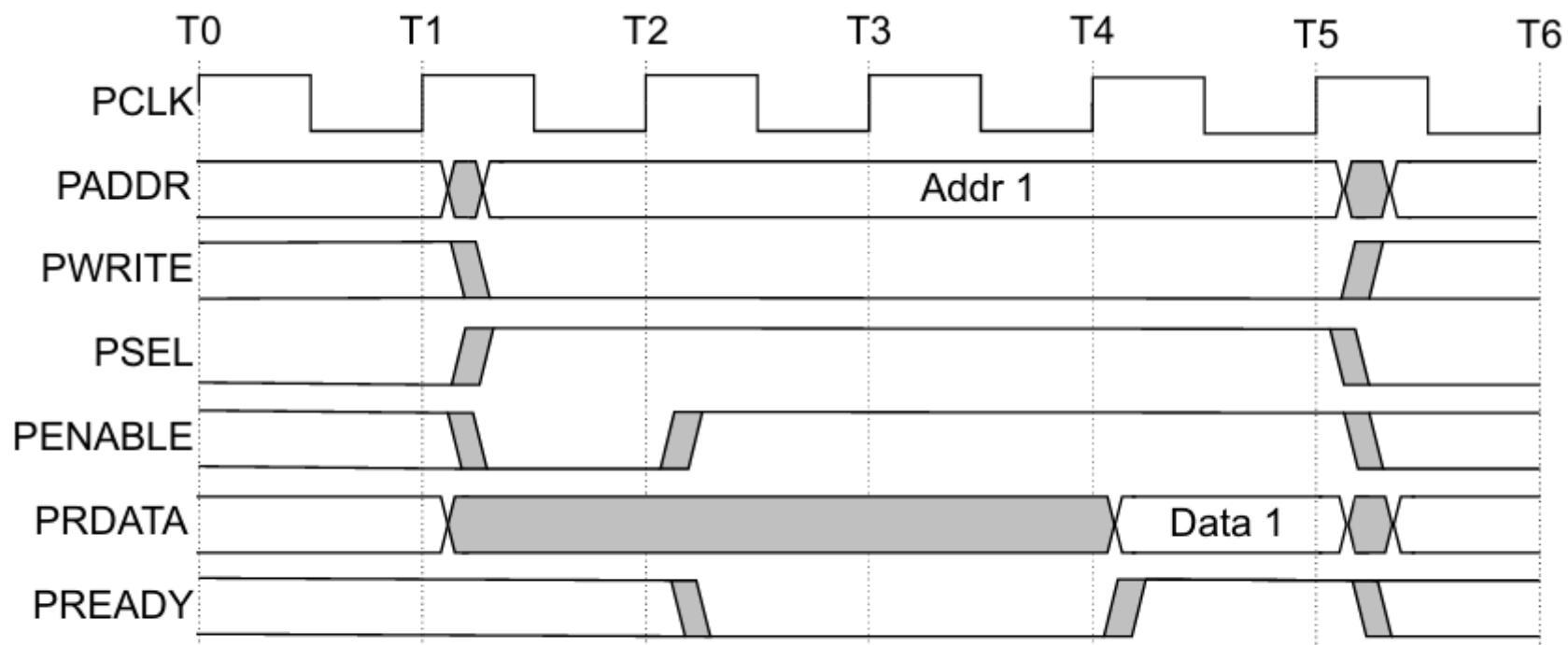
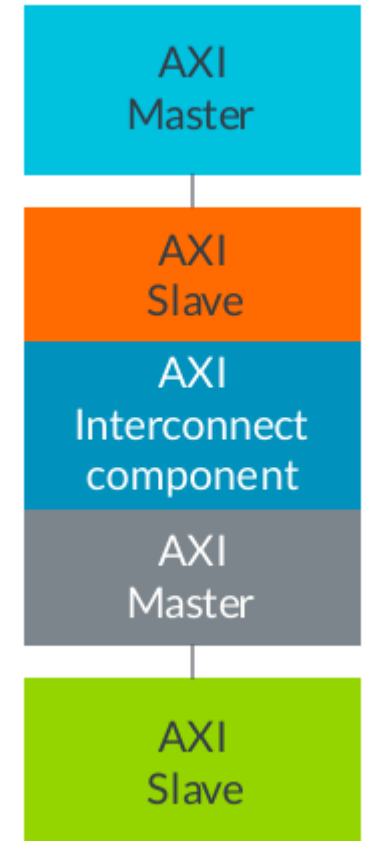
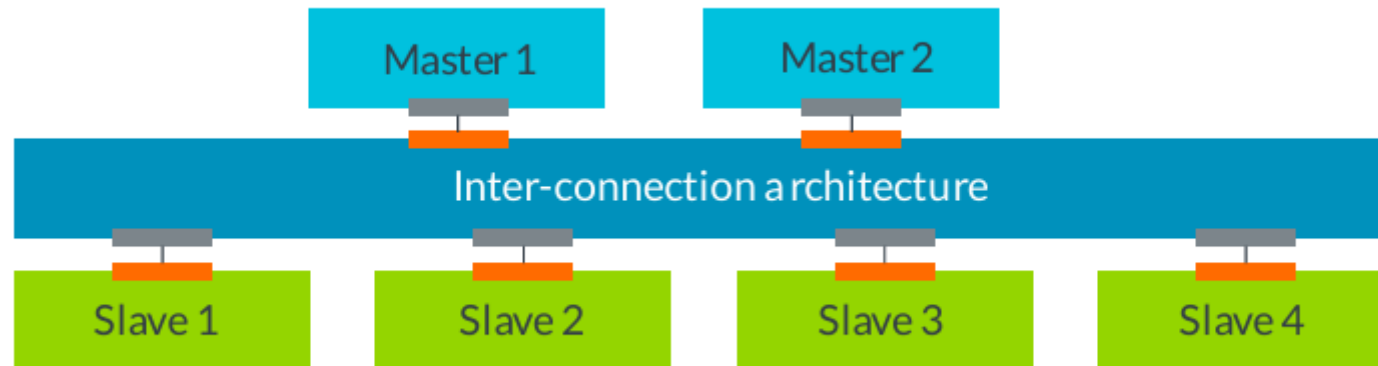


Figure 3-5 Read transfer with wait states

AXI (AMBA eXtensible Interface)

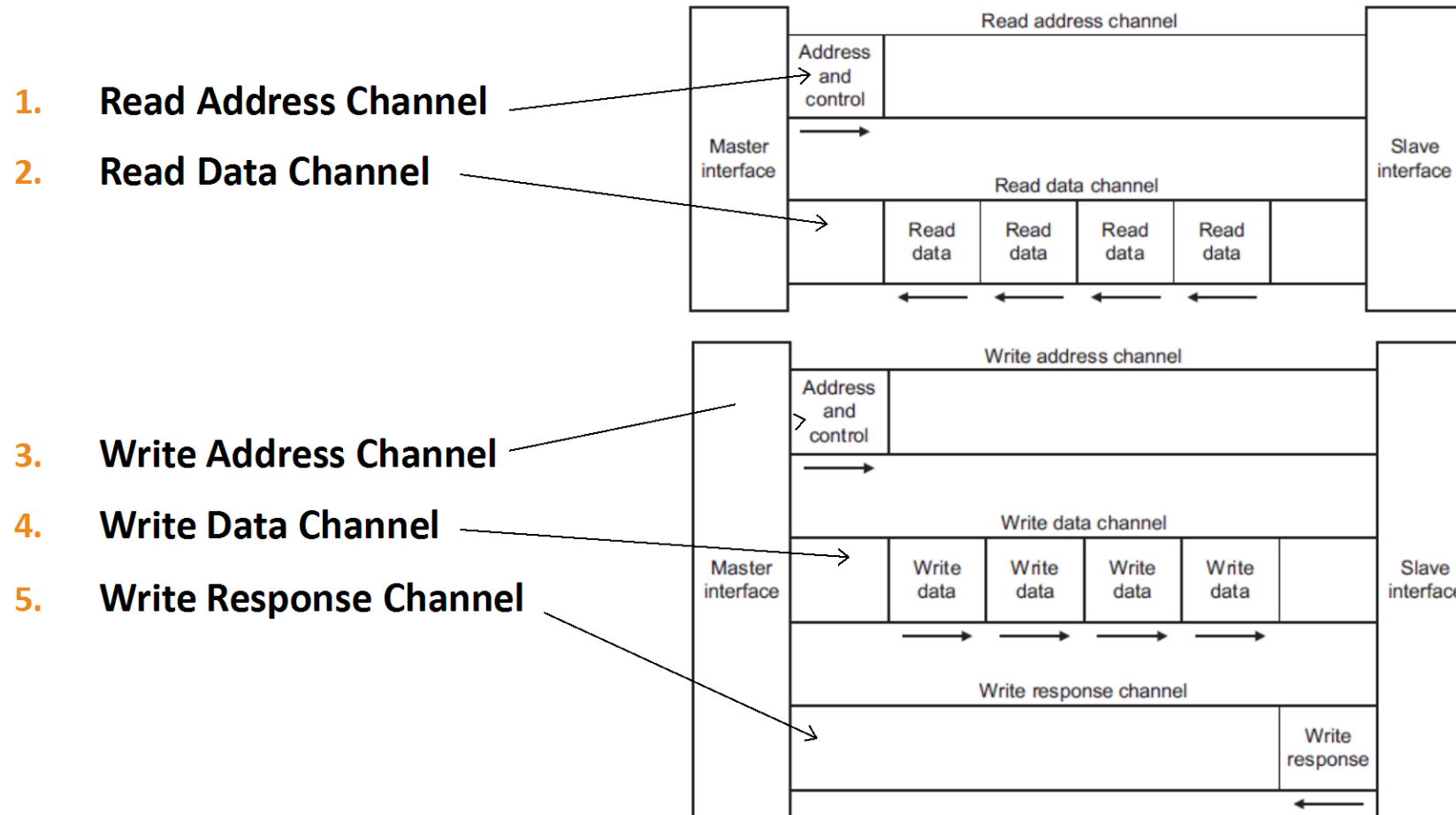
- High-bandwidth, high-performance
- Independent read and write channels
- Multiple outstanding addresses
- No strict timing relationship between address and data operations
- Out-of-order transaction completion support



Basic AXI Signaling – 5 Channels

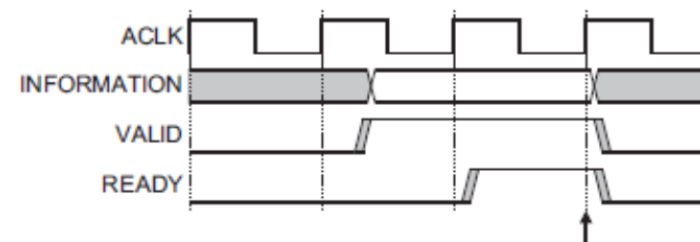
AXI, at the highest level consists of the 5 channels shown.

Each channel is independent. And as you'll see, whether its AXI4, AXI4-Lite or AXI4-Stream, the interfaces are effectively the same.

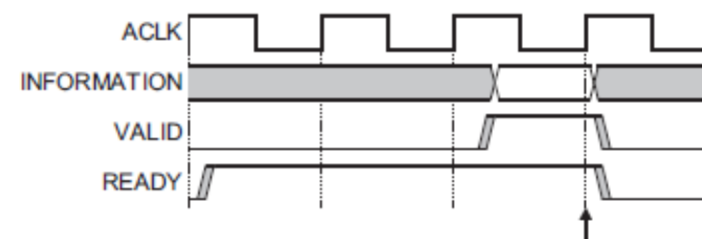


AXI Interface: Handshaking

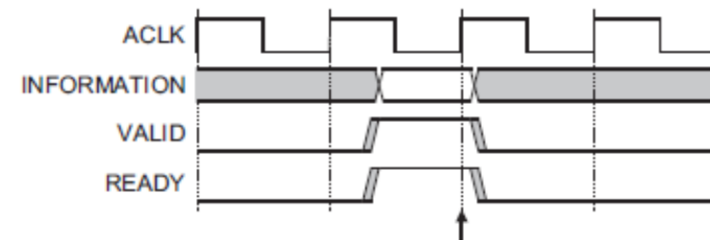
- **AXI uses a valid/ready handshake**
- **Each channel has its own valid/ready**
 - Address (read/write)
 - Data (read/write)
 - Response (write only)
- **Flexible signaling functionality**
 - Inserting wait states
 - Always ready
 - Same cycle acknowledge



Inserting Wait States



Always Ready



Same Cycle Acknowledge

AXI Signals

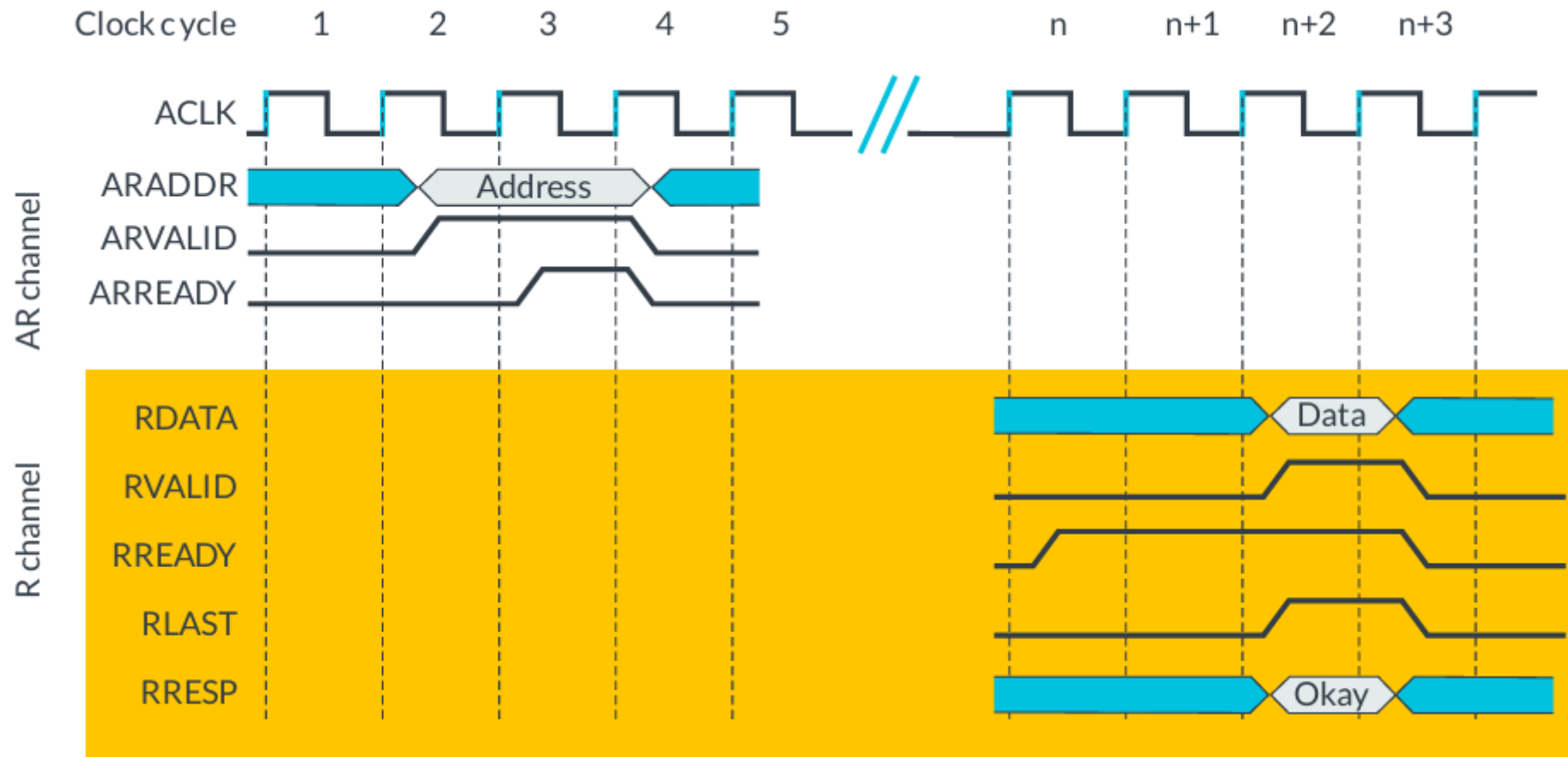
- AW for signals on the Write Address channel
- AR for signals on the Read Address channel
- W for signals on the Write Data channel
- R for signals on the Read Data channel
- B for signals on the Write Response channel

Signals	Read Address	Read Data	Write Address	Write Data	Write Response
HANDSHAKE	ARVALID ARREADY	RVALID RREADY	AWVALID AWREADY	WVALID WREADY	BVALID BREADY
INFORMATION	ARADDR	RDATA RLAST	AWADDR	WDATA WLAST	BRESP
GLOBAL	ACLK, ARESETn				

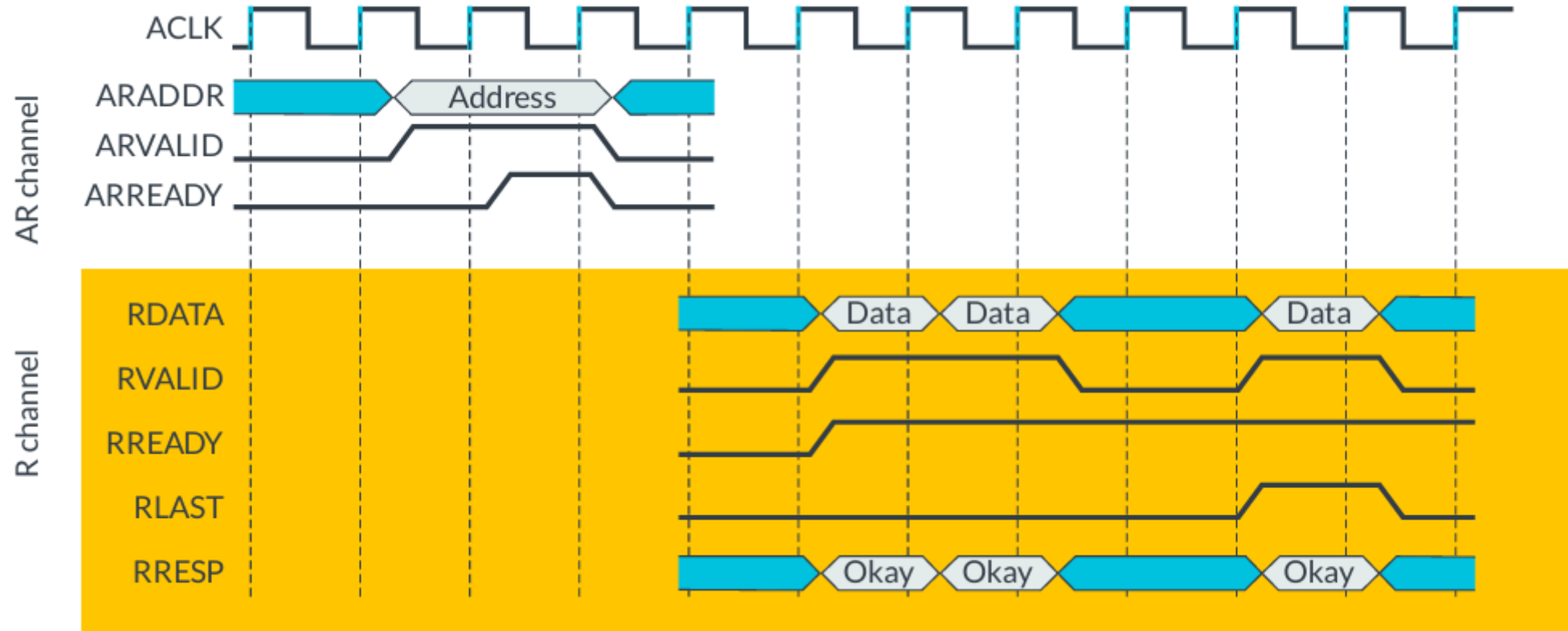
AXI Properties

- Bursts up to 256 data beats
- Data width up to 1024 bits
- Address width up to 64 bits

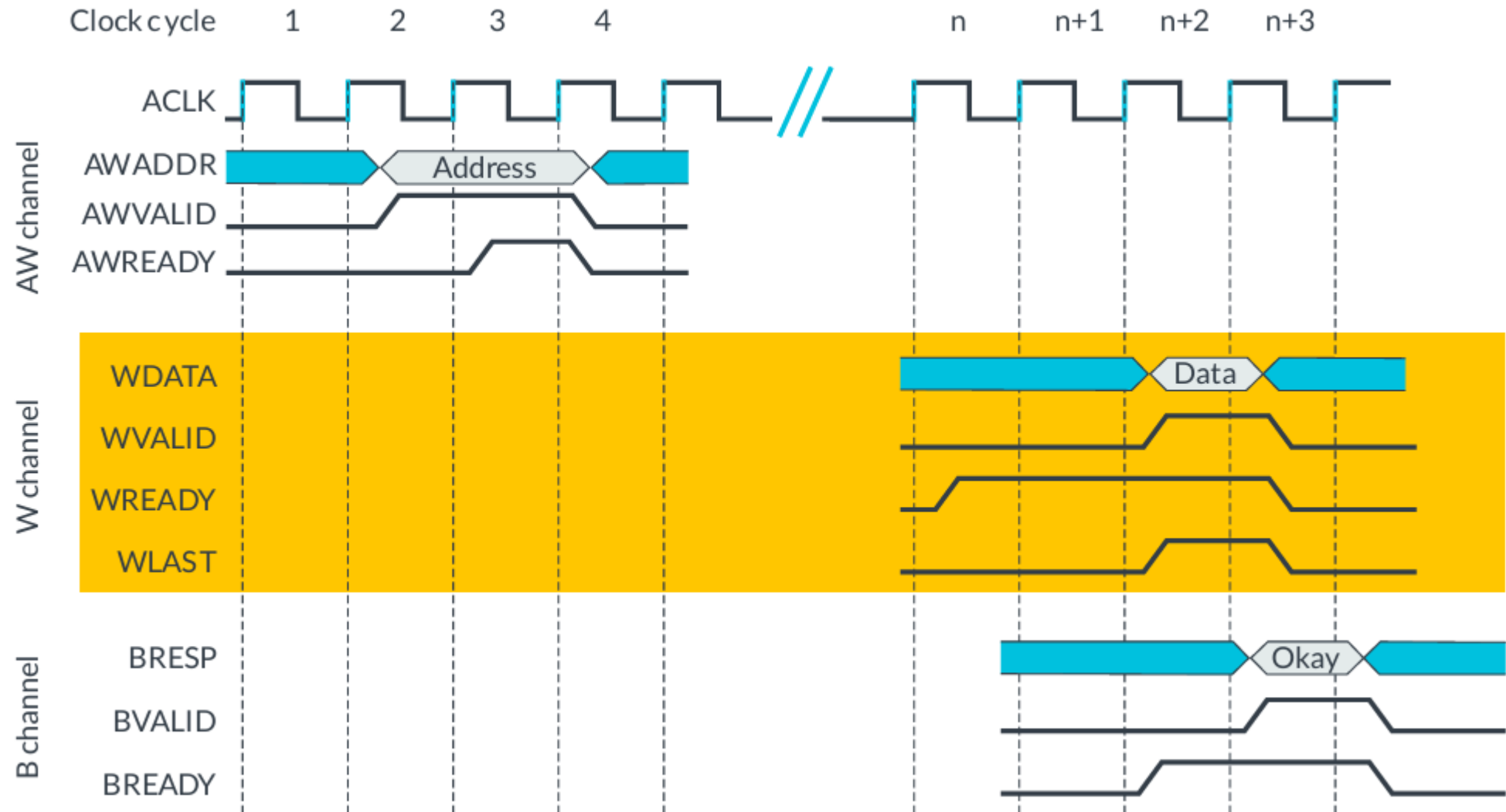
AXI Read Transaction



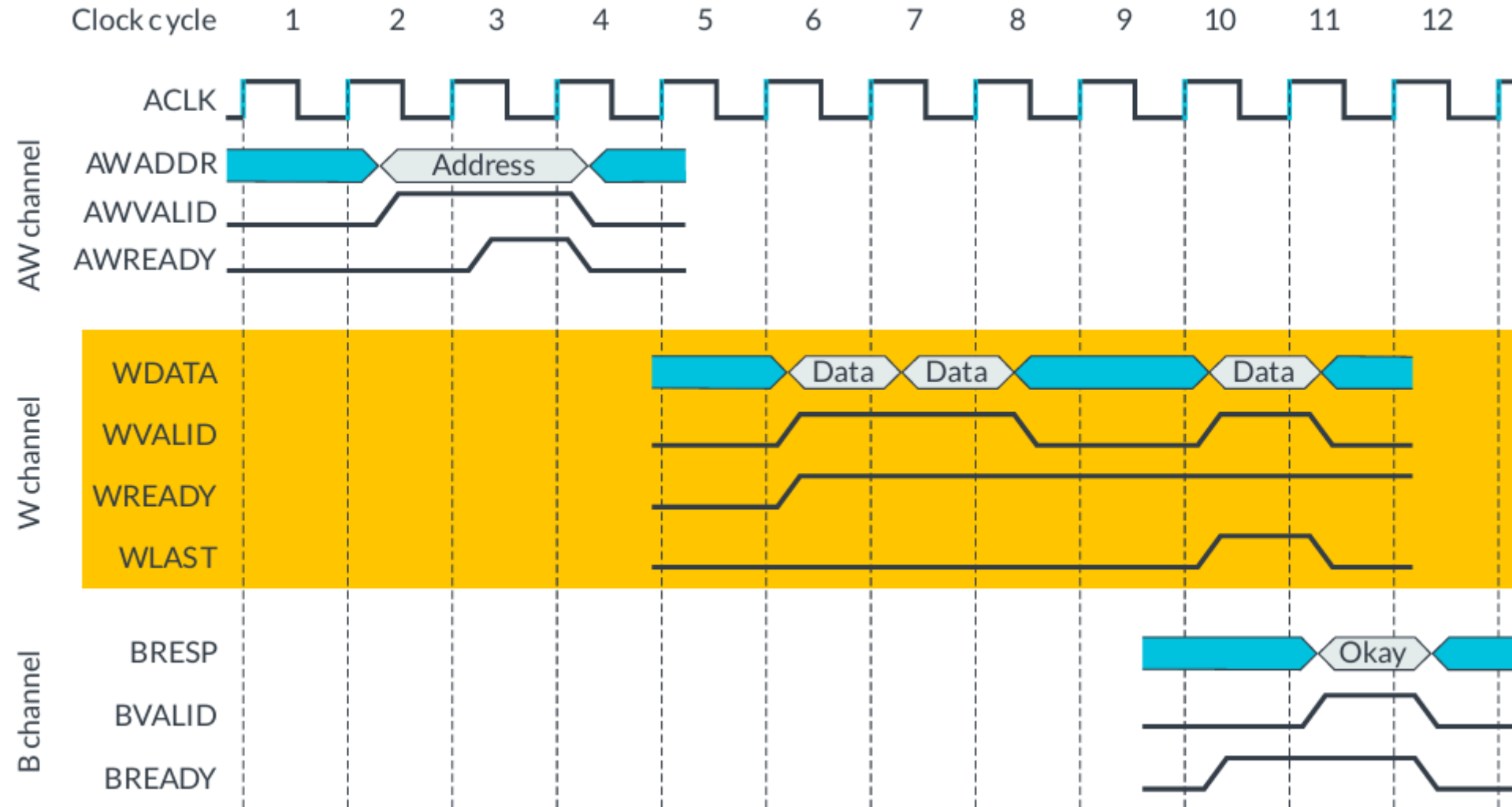
AXI Read Burst Transaction



AXI Write Transaction



AXI Write Burst Transaction



Closing Slides

Chipllets – The Future of Integration

- A chiplet is a small silicon die designed to be integrated with other dies in the same package
- Reasons:
 - Yield and cost
 - Mix process technologies
 - Scalability and reuse
 - Reticle/die-size limits
- System-on-Package (SoP)
- New challenges
 - Die-to-die communication (Protocols like UCIe)
 - Thermal and power delivery
- Architectures
 - 2.5D, 3D

